Natural Language Processing

Prof. Ahmed Guessoum
The National Higher School of AI

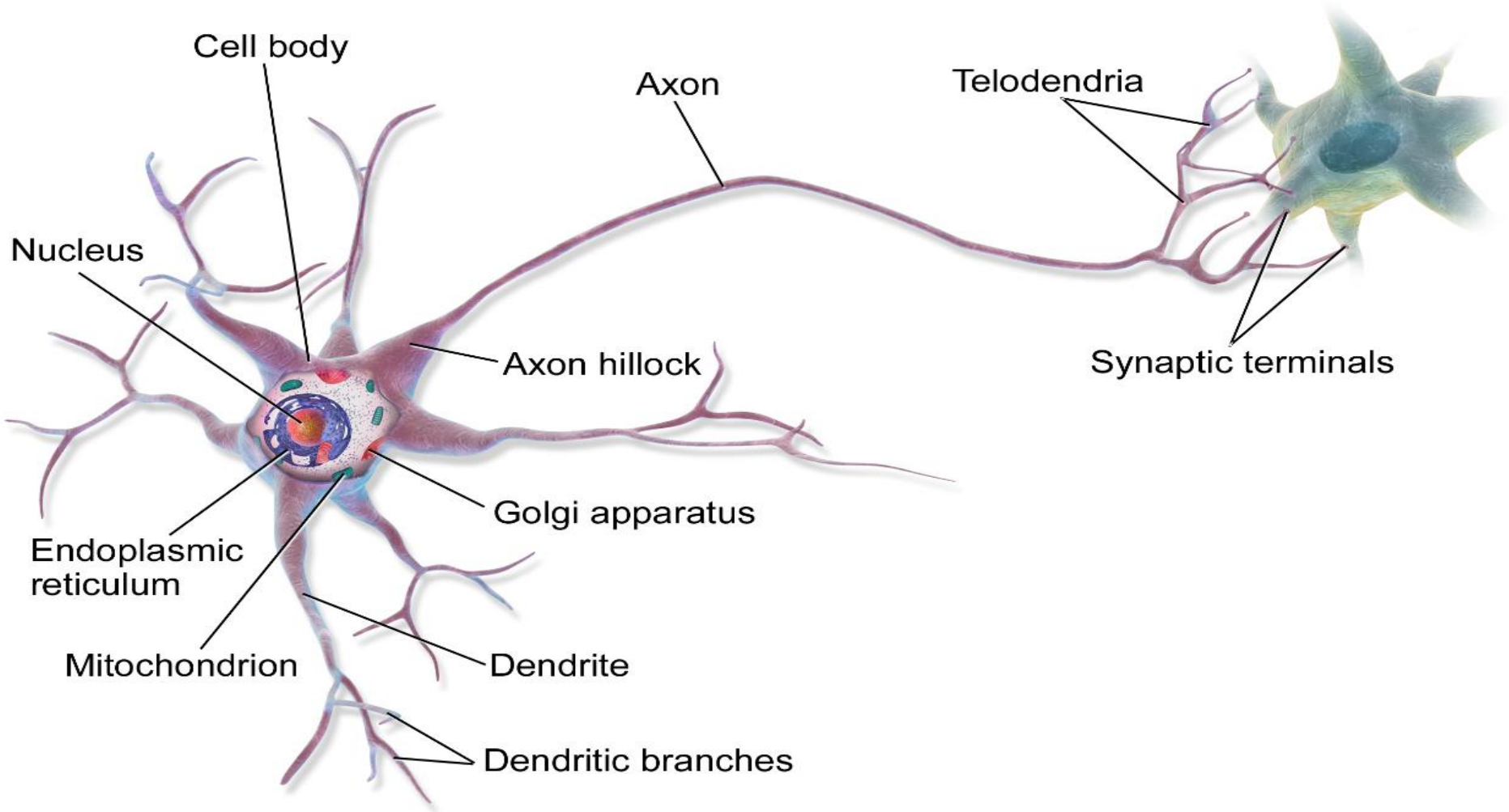
Chapter 7

Neural Networks & Neural Language Models

Units in Neural Networks

(Material from ML: Individual Reading)

This is in your brain



By BruceBlaus - Own work, CC BY 3.0, https://commons.wikimedia.org/w/index.php?curid=28761830

Neural Network Unit

This is not in your brain

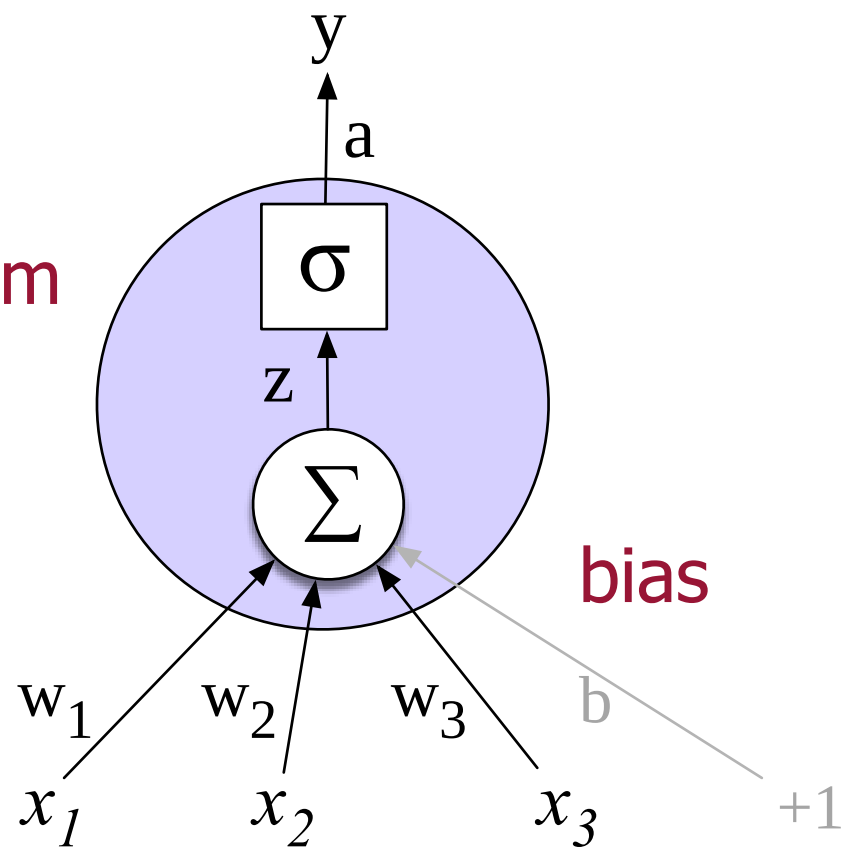
Output value

Non-linear transform

Weighted sum

Weights

Input layer



Neural unit

- Take weighted sum of inputs, plus a bias

$$z = \left(\sum_{i=1}^{n} w_i x_i \right) + b$$

$$z = w \cdot x + b$$

- Instead of just using z, we'll apply a nonlinear activation function f:

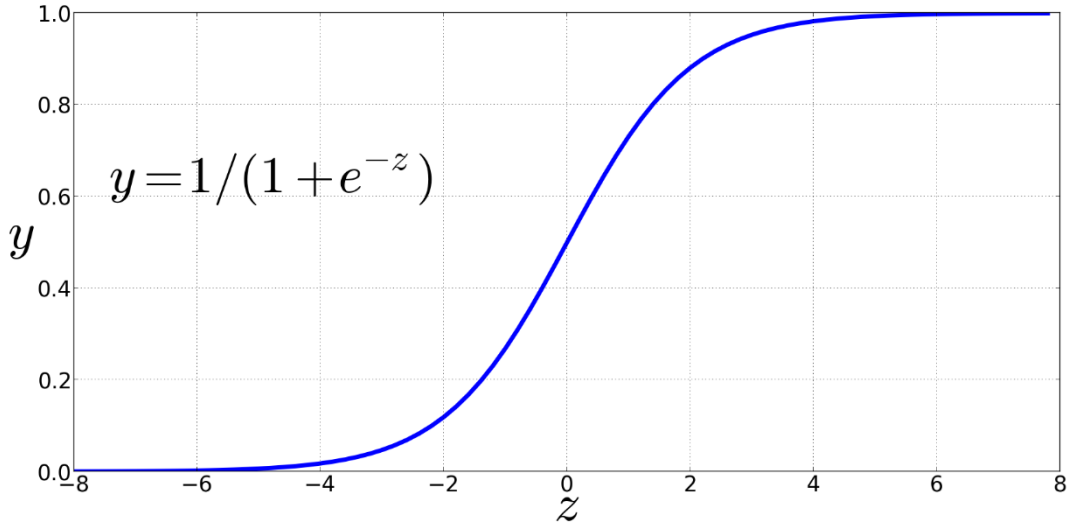
$$y = a = f(z)$$

Non-Linear Activation Functions

We're already seen the sigmoid for logistic regression:

Sigmoid

$$y = s(z) = \frac{1}{1 + e^{-z}}$$



Final function the unit is computing

$$y = s(w \cdot x + b) = \frac{1}{1 + \exp(-(w \cdot x + b))}$$

Final unit again

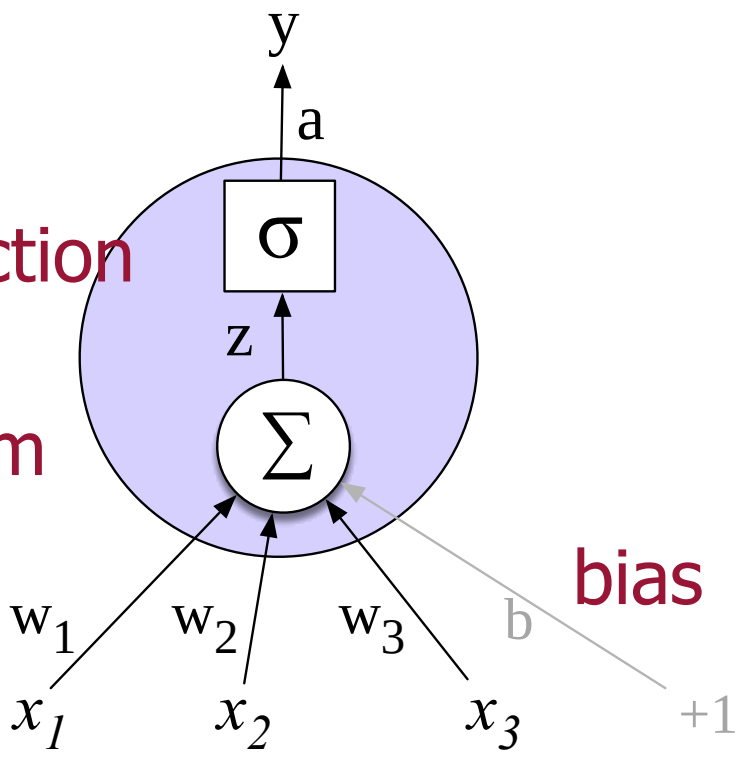
Output value

Non-linear activation function

Weighted sum

Weights

Input layer



An example

- *Suppose a unit has:*
- $w = [0.2, 0.3, 0.9]$
- $b = 0.5$
- What happens with input x:
- $x = [0.5, 0.6, 0.1]$

$$y = s(w \cdot x + b) =$$

An example

- *Suppose a unit has:*
  - $w = [0.2, 0.3, 0.9]$
  - $b = 0.5$
- What happens with the following input x?

$$x = [0.5, 0.6, 0.1]$$

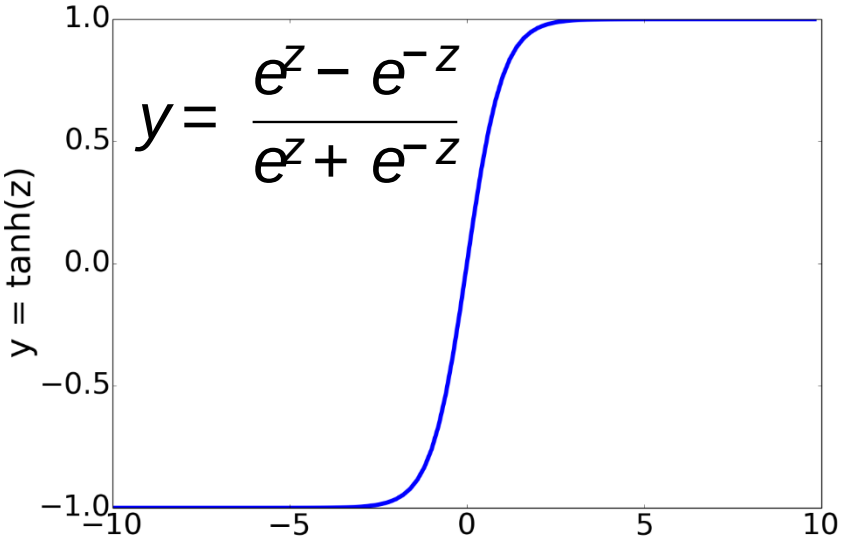
$$y = s(w \cdot x + b) = \frac{1}{1 + e^{-(w \cdot x + b)}} =$$

An example

- *Suppose a unit has:*
- $w = [0.2, 0.3, 0.9]$
- $b = 0.5$
- What happens with input x:
- $x = [0.5, 0.6, 0.1]$

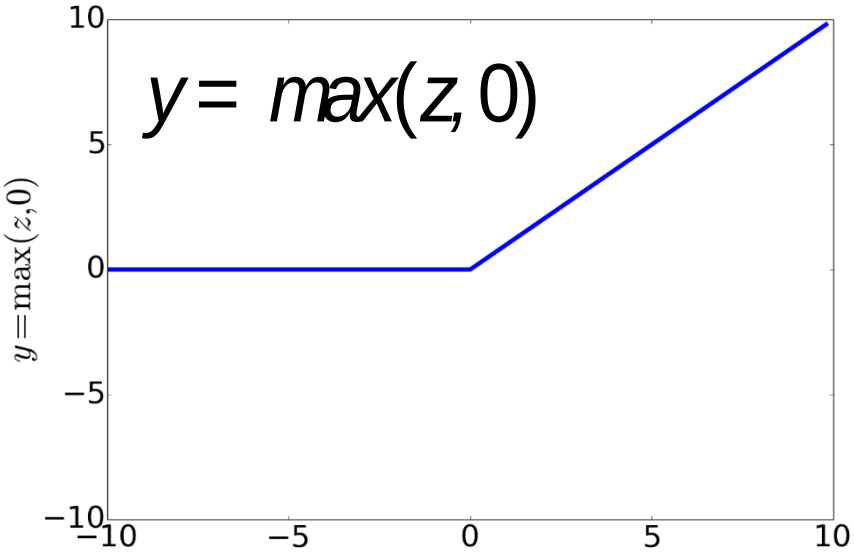
$$y = s(w \cdot x + b) = \frac{1}{1 + e^{-(w \cdot x + b)}} = \frac{1}{1 + e^{-0.87}} = .70$$

Non-Linear **Activation** Functions besides sigmoid



tanh

Most Common:



ReLU
Rectified Linear Unit

The XOR problem

(Material from ML: Individual Reading)

The XOR problem

Minsky and Papert (1969)

- Can neural units compute simple functions of input?

AND			OR			XOR		
x1	x2	y	x1	x2	y	x1	x2	y
0	0	0	0	0	0	0	0	0
0	1	0	0	1	1	0	1	1
1	0	0	1	0	1	1	0	1
1	1	1	1	1	1	1	1	0

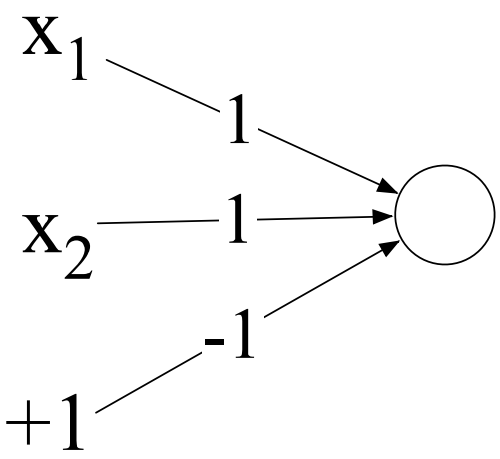
Perceptrons

- A very simple neural unit
- Binary output (0 or 1)
- No non-linear activation function

$$y = \begin{cases} 0, & \text{if } w \cdot x + b \leq 0 \\ 1, & \text{if } w \cdot x + b > 0 \end{cases}$$

Easy to build AND or OR with perceptrons

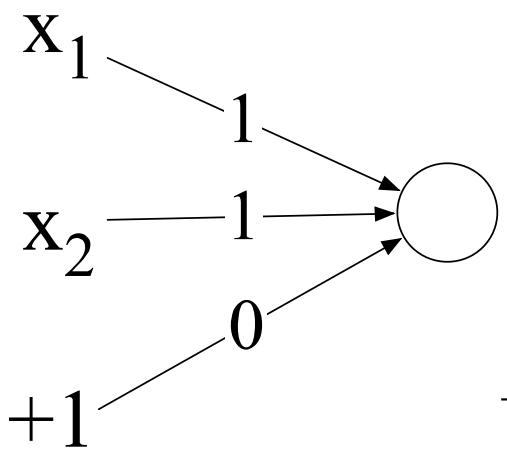
$$y = \begin{cases} 0, & \text{if } w \cdot x + b \leq 0 \\ 1, & \text{if } w \cdot x + b > 0 \end{cases}$$



AND

AND

x1	x2	y
0	0	0
0	1	0
1	0	0
1	1	1



OR

OR

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	1

XOR & perceptrons

- Not possible to capture XOR with perceptrons
- Try for yourself!

Why?

- Perceptrons are linear classifiers
- Perceptron equation given x_1 and x_2, is the equation of a line

$$w_1 x_1 + w_2 x_2 + b = 0$$

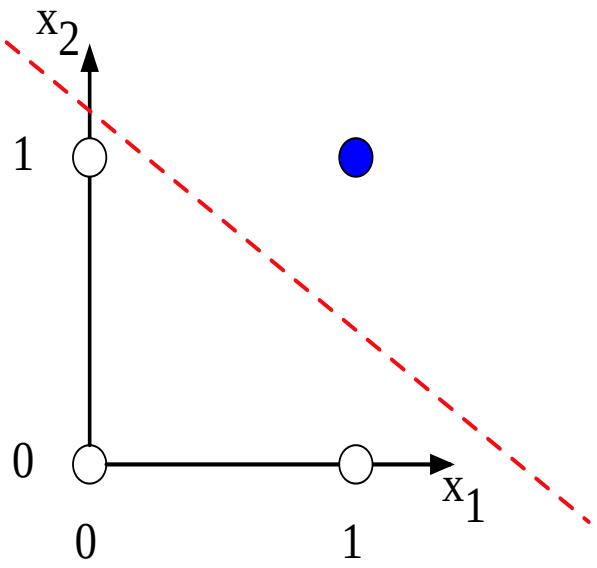
- (in standard linear format: $x_2 = (-w_1/w_2)x_1 + (-b/w_2)$)

- This line acts as a **decision boundary**

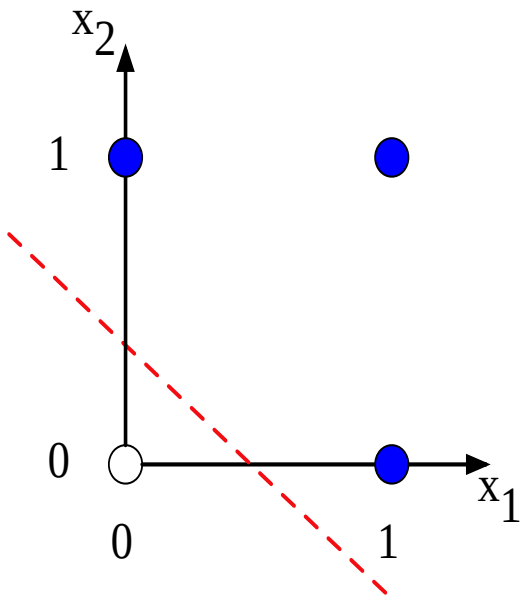
  0 if input is on one side of the line

  1 if on the other side of the line

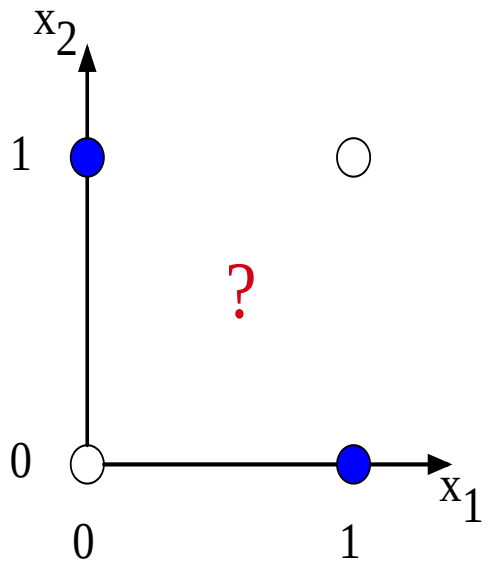
Decision boundaries



a) x_1 AND x_2



b) x_1 OR x_2



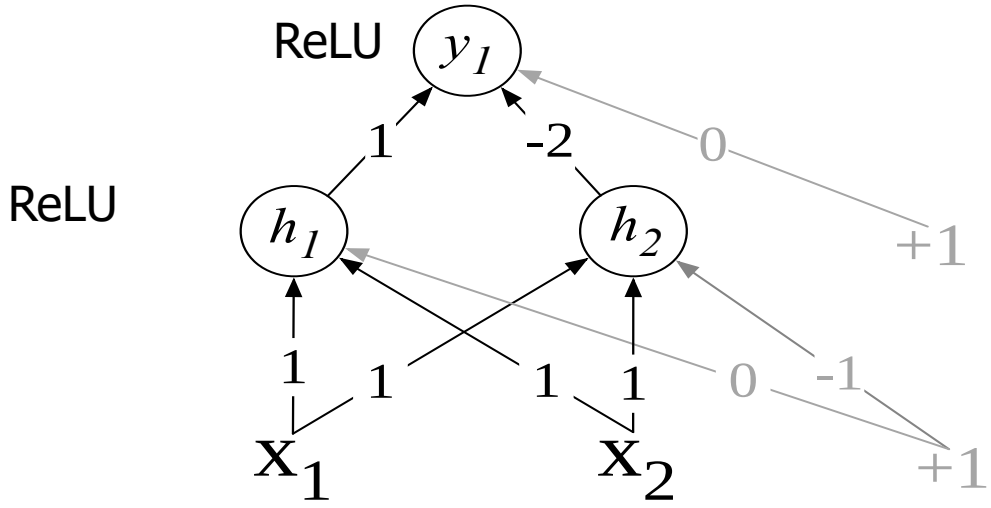
c) x_1 XOR x_2

XOR is not a **linearly separable** function!

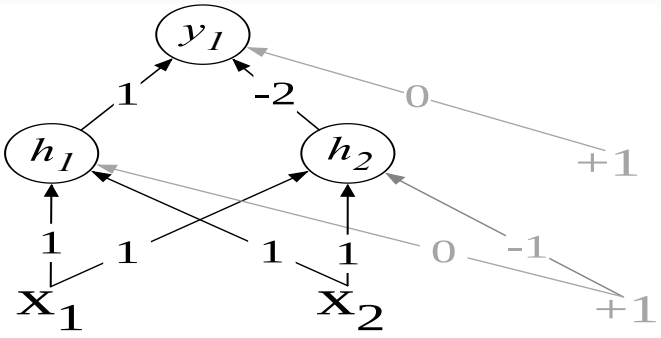
Solution to the XOR problem

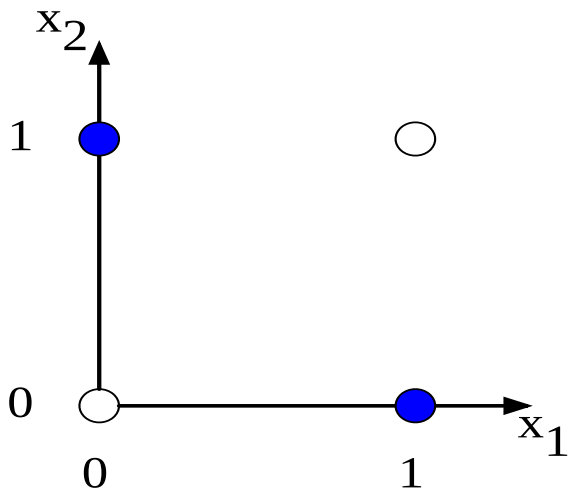
- XOR **can't** be calculated by a single perceptron
- XOR **can** be calculated by a layered network of units.

XOR		
x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

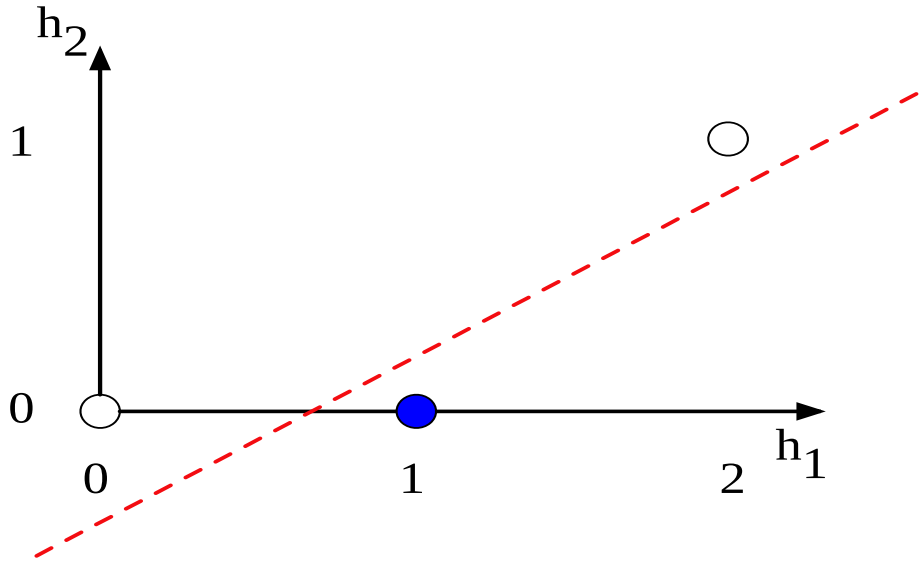


The hidden representation h





a) The original x space



b) The new (linearly separable) h space

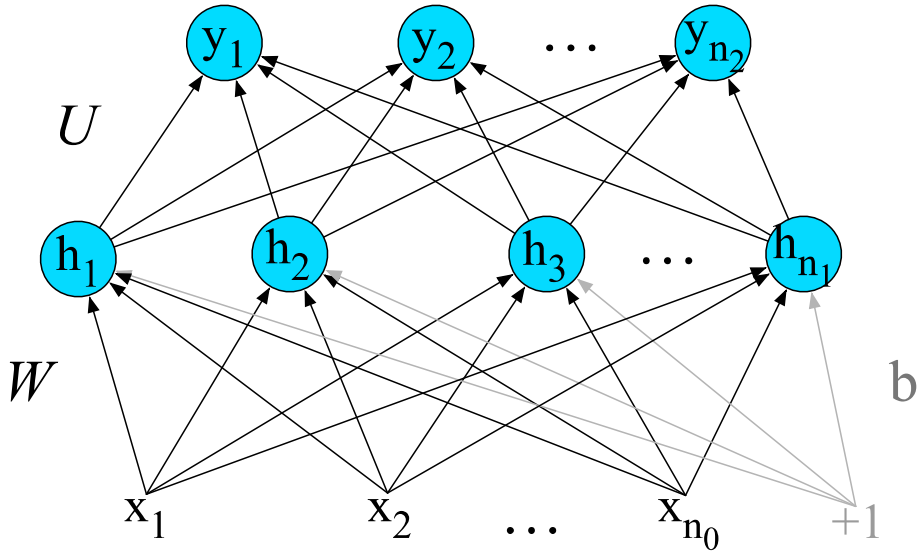
(With learning: hidden layers will learn to form useful representations)

Feedforward Neural Networks

(Material from ML: Individual Reading)

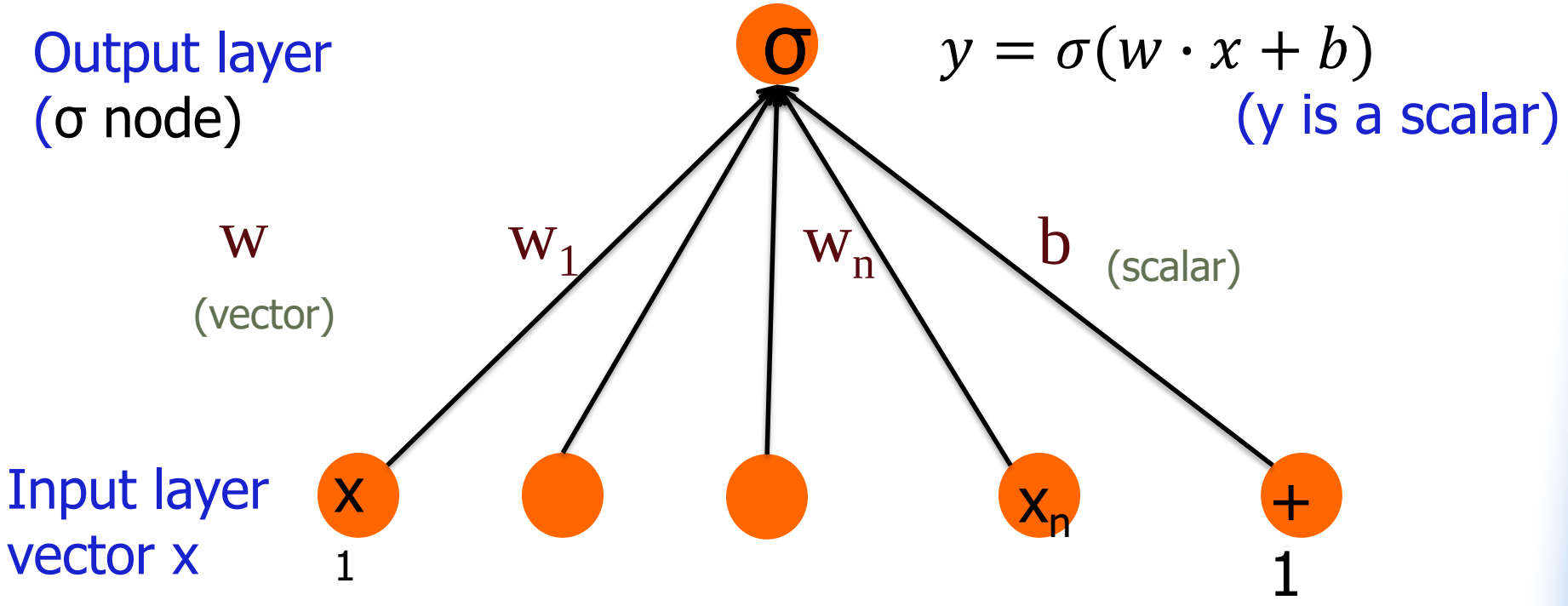
Feedforward Neural Networks

- Can also be called **multi-layer perceptrons** (or **MLPs**) for historical reasons



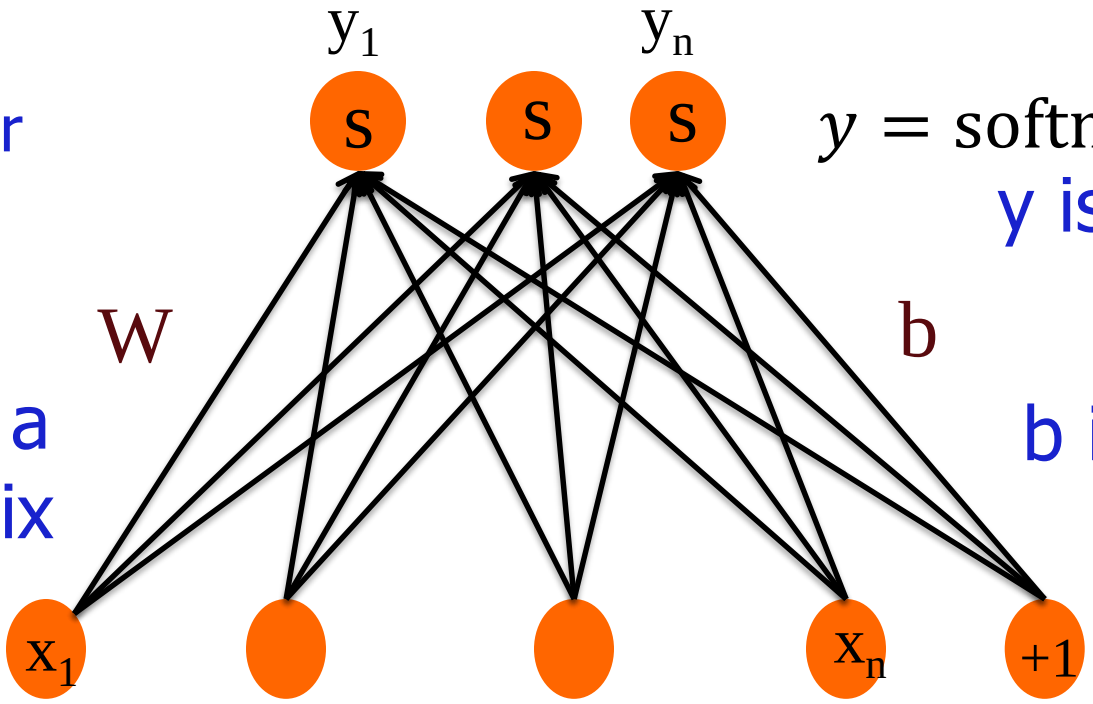
Binary Logistic Regression as a 1-layer Network

(we don't count the input layer in counting layers!)



Multinomial Logistic Regression as a 1-layer Network

Fully connected single layer network



Output layer (softmax nodes)

$y = \text{softmax}(Wx + b)$

y is a vector

W is a matrix

b is a vector

Input layer scalars

Reminder: softmax: a generalization of sigmoid

- For a vector z of dimensionality k, the softmax is:

$$\text{softmax}(z) = \left[\frac{\exp(z_1)}{\sum_{i=1}^{k} \exp(z_i)}, \frac{\exp(z_2)}{\sum_{i=1}^{k} \exp(z_i)}, \ldots, \frac{\exp(z_k)}{\sum_{i=1}^{k} \exp(z_i)} \right]$$

$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_{j=1}^{k} \exp(z_j)} \quad 1 \leq i \leq k$$

- Example: $z = [0.6, 1.1, -1.5, 1.2, 3.2, -1.1]$

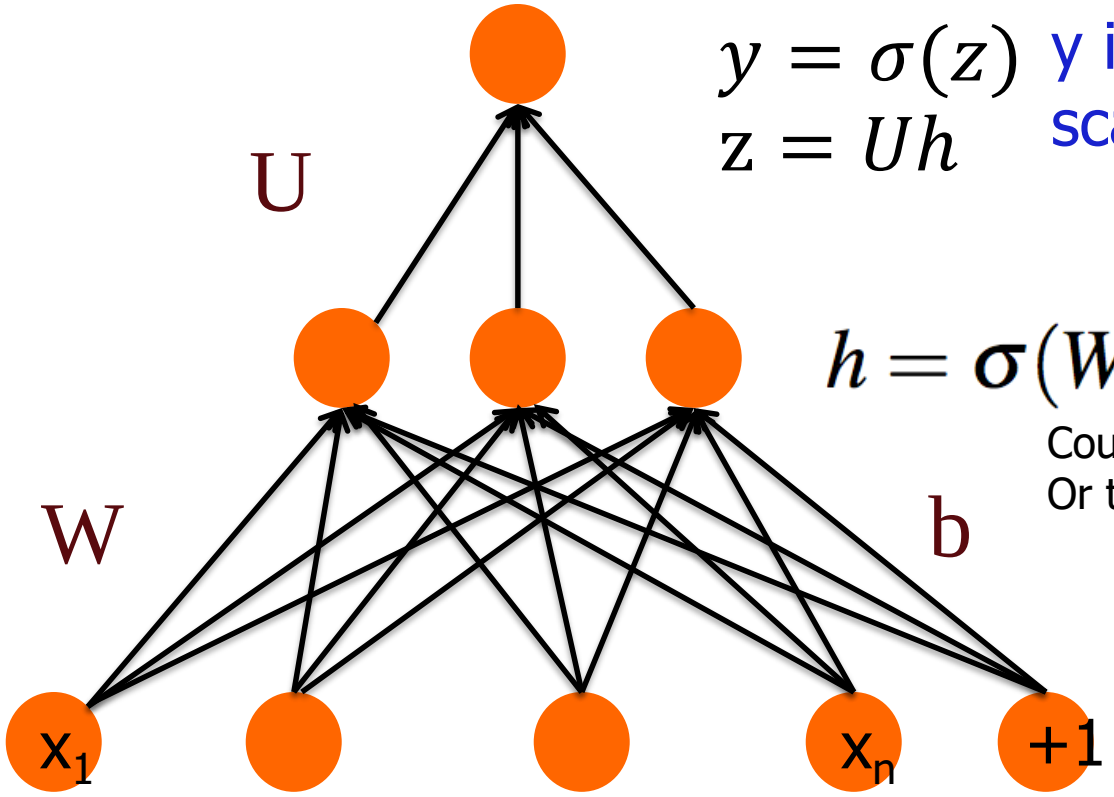
$$\text{softmax}(z) = [0.055, 0.090, 0.006, 0.099, 0.74, 0.010]$$

Two-Layer Network with scalar output

Output layer (σ node)

hidden units (σ nodes)

Input layer (vector)



$$y = \sigma(z)$$
$$z = Uh$$

y is a scalar

$$h = \sigma(Wx + b)$$

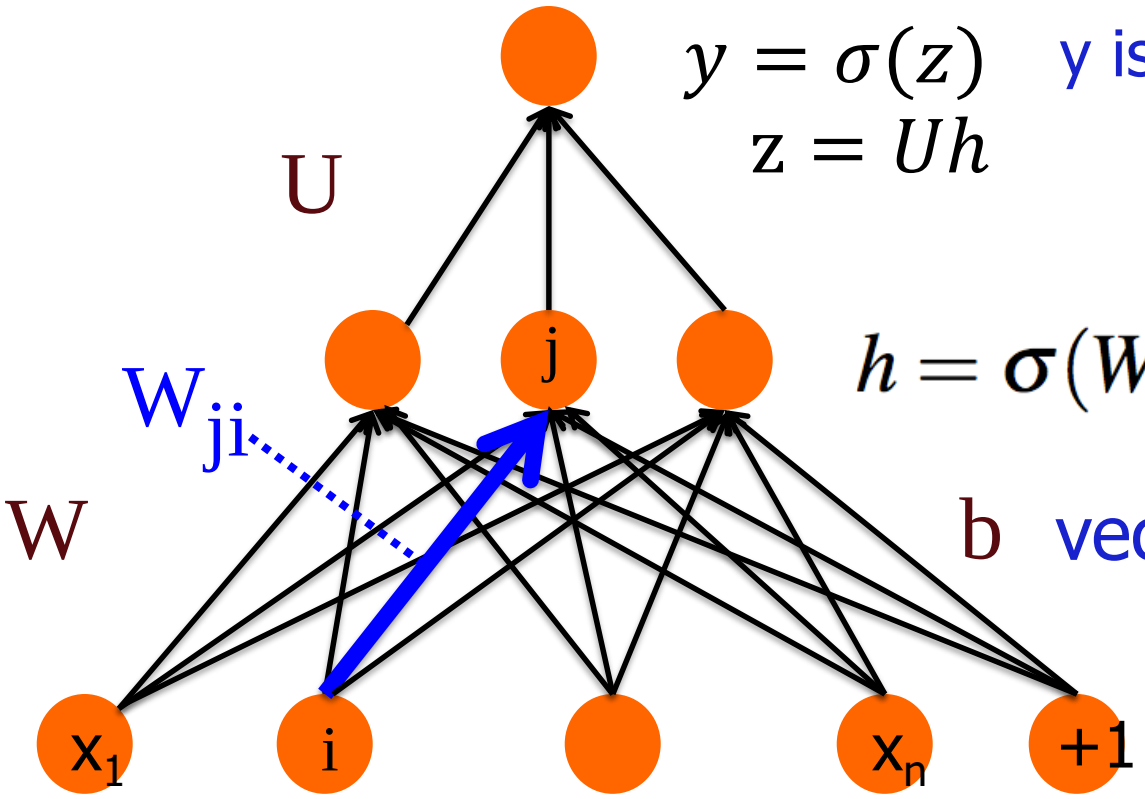
Could be ReLU
Or tanh

Two-Layer Network with scalar output

Output layer (σ node)

hidden units (σ node)

Input layer (vector)

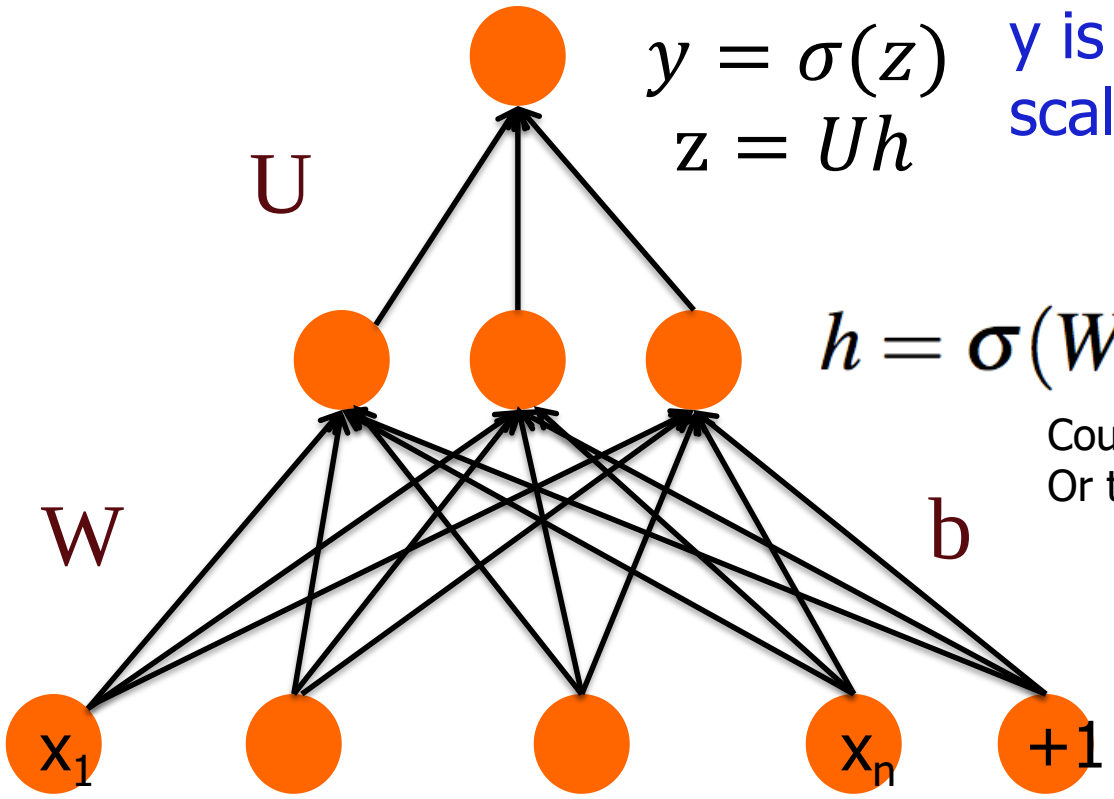


Two-Layer Network with scalar output

Output layer (σ node)

hidden units (σ node)

Input layer (vector)

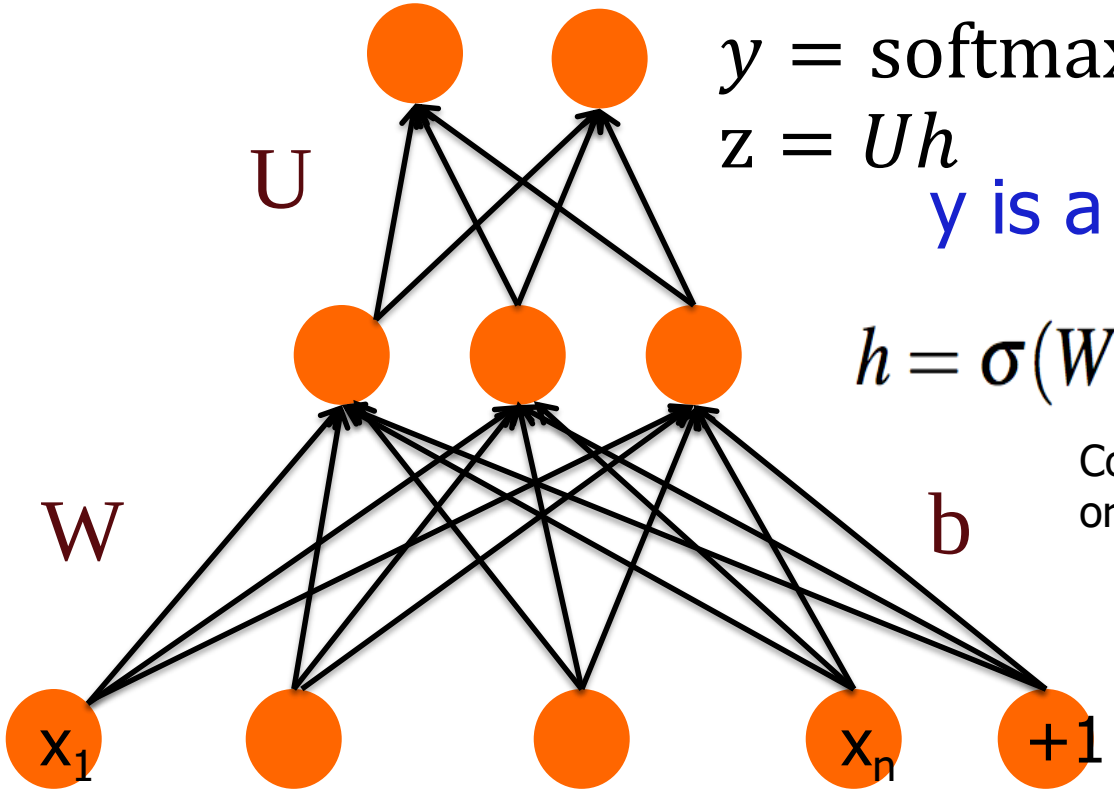


Two-Layer Network with softmax output

Output layer (σ node)

hidden units (σ node)

Input layer (vector)



$$y = \text{softmax}(z)$$

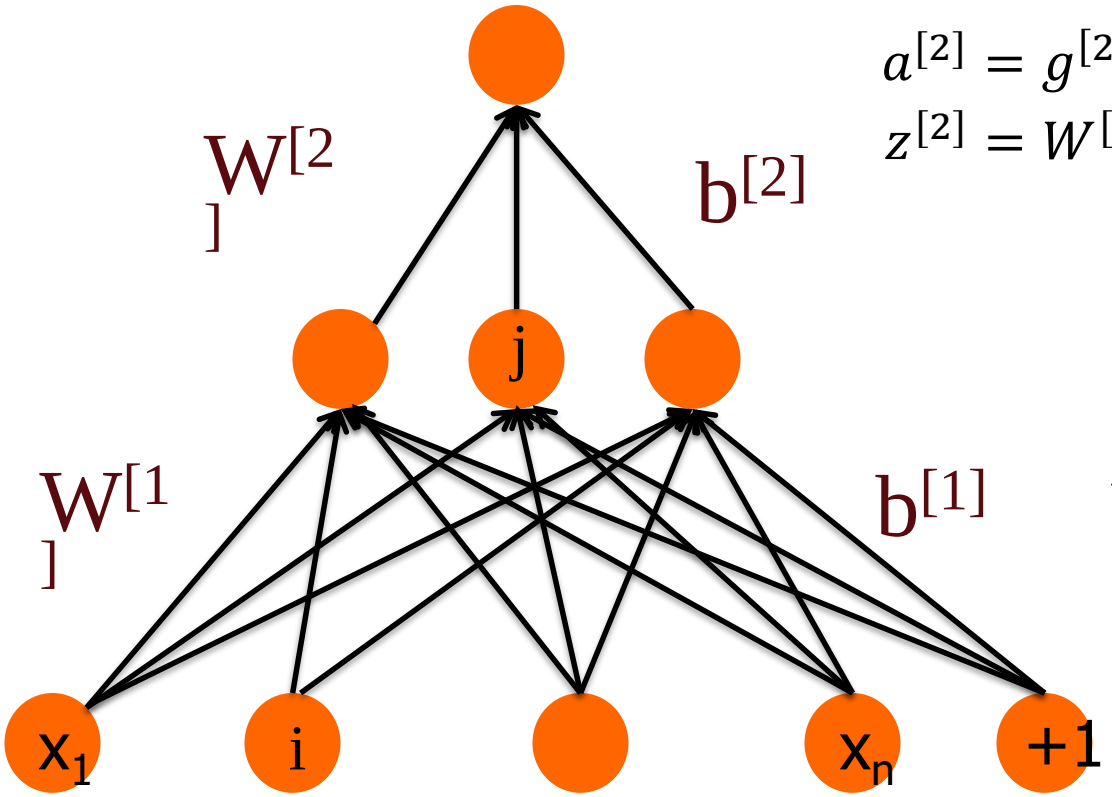
$$z = Uh$$

y is a vector

$$h = \sigma(Wx + b)$$

Could be ReLU or tanh

Multi-layer Notation



$$y = a^{[2]}$$

$$a^{[2]} = g^{[2]}(z^{[2]})$$ sigmoid or softmax

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

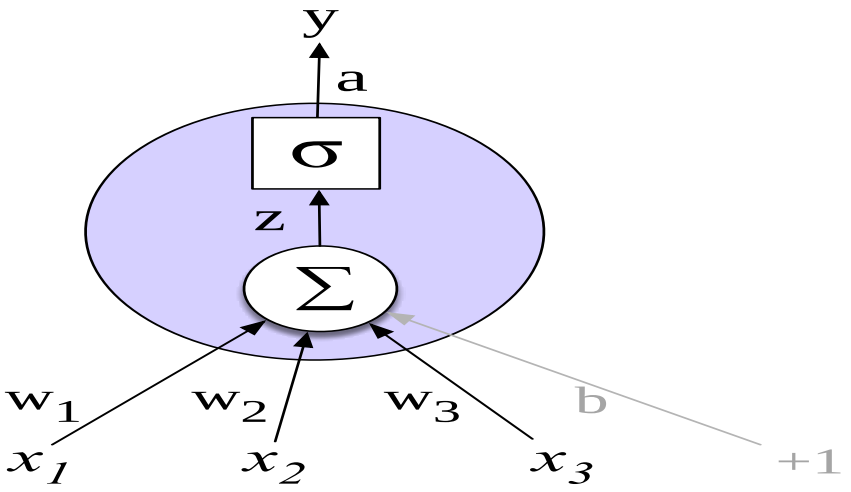
$$a^{[1]} = g^{[1]}(z^{[1]})$$ ReLU

$$z^{[1]} = W^{[1]}a^{[0]} + b^{[1]}$$

$a^{[0]}$

Multi Layer Notation

$$
\begin{aligned}
z^{[1]} &= W^{[1]} a^{[0]} + b^{[1]} \\
a^{[1]} &= g^{[1]}(z^{[1]}) \\
z^{[2]} &= W^{[2]} a^{[1]} + b^{[2]} \\
a^{[2]} &= g^{[2]}(z^{[2]}) \\
\hat{y} &= a^{[2]}
\end{aligned}
$$



for i **in** 1..n

$$
\begin{aligned}
z^{[i]} &= W^{[i]} a^{[i-1]} + b^{[i]} \\
a^{[i]} &= g^{[i]}(z^{[i]}) \\
\hat{y} &= a^{[n]}
\end{aligned}
$$

Replacing the bias unit

- Let's switch to a notation without the bias unit
- Just a notational change

1. Add a dummy node $a_0=1$ to each layer
2. Its weight w_0 will be the bias
3. So input layer $a^{[0]}_0=1$,
   - And $a^{[1]}_0=1$, $a^{[2]}_0=1$,…

Replacing the bias unit

- Instead of this:

$$x = x_1, x_2, \ldots, x_{n0}$$

$$h = \sigma(Wx + b)$$

$$h_j = \sigma\left(\sum_{i=1}^{n_0} W_{ji} x_i + b_j\right)$$

We'll do

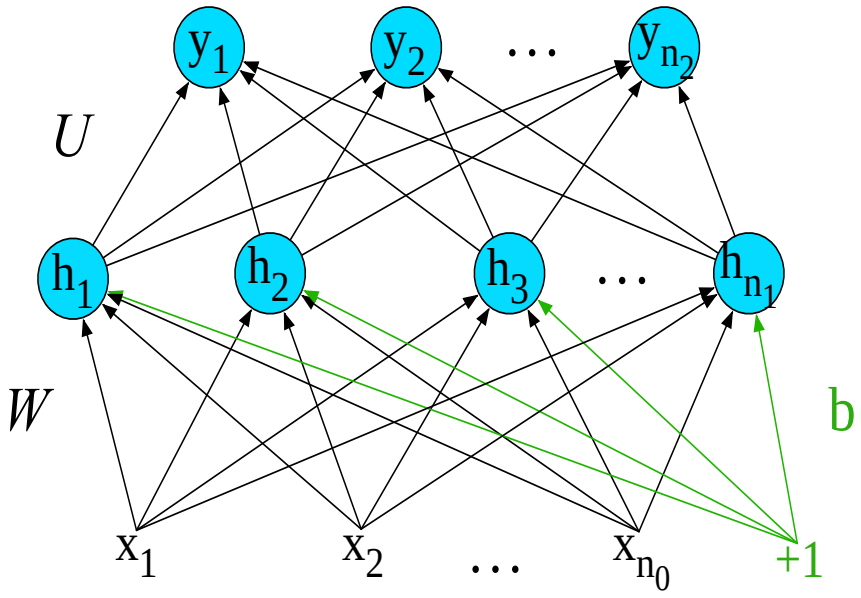
$$x = x_0, x_1, x_2, \ldots, x_{n0}$$

$$h = \sigma(Wx)$$

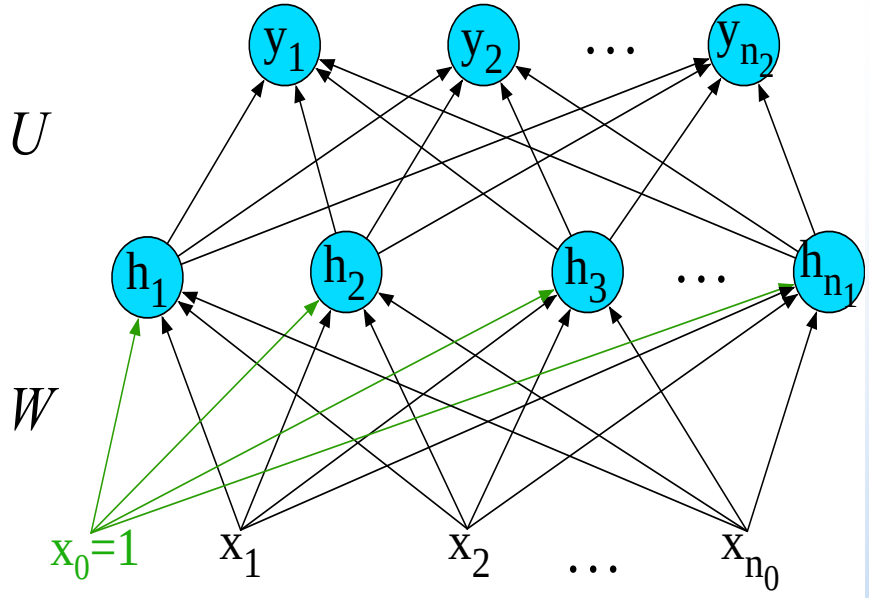
$$\sigma\left(\sum_{i=0}^{n_0} W_{ji} x_i\right)$$

Replacing the bias unit

Instead of:



We'll do this:



Applying Feedforward Neural Networks to NLP Tasks

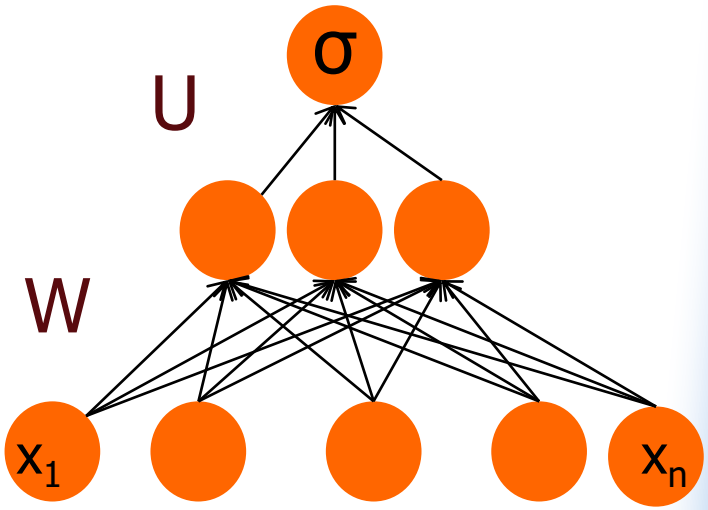
Use cases for feedforward networks

- Let's consider 2 (simplified) sample tasks:
  1. Text classification
  2. Language modeling

- State of the art systems use more powerful neural architectures, but simple models are useful to consider!

Classification: Sentiment Analysis

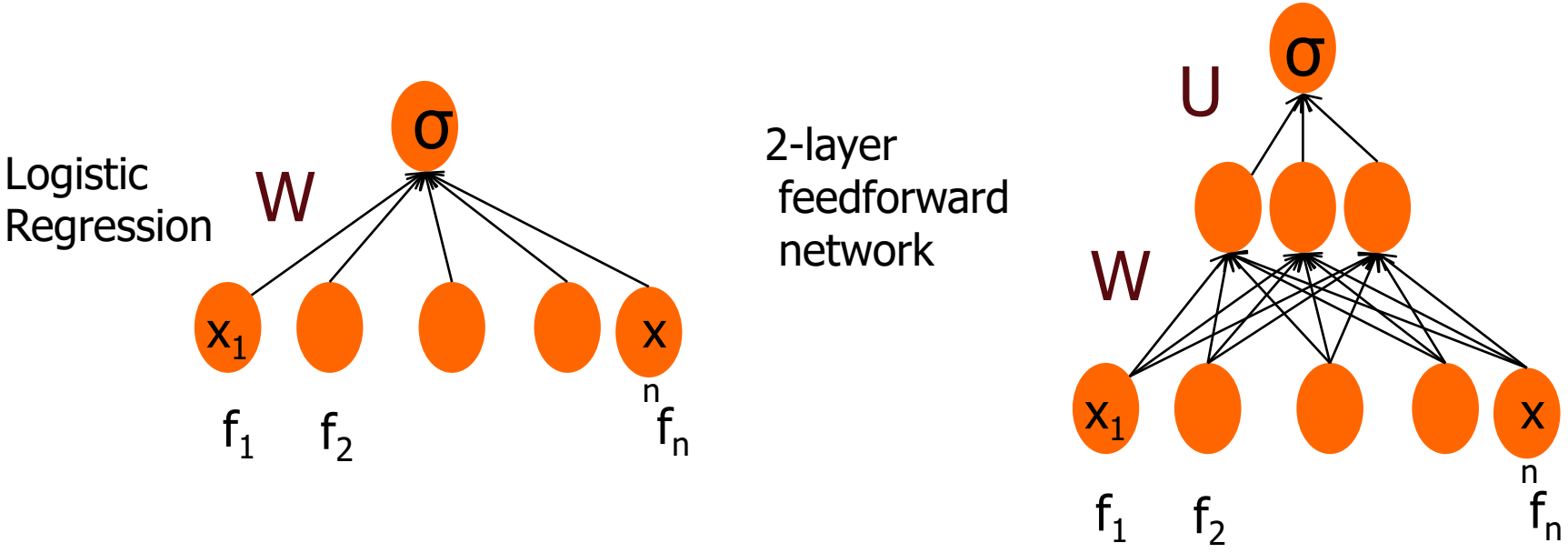
- We could do exactly what we did with logistic regression
- Input layer: binary features as before
- Output layer is 0 or 1



Sentiment Features

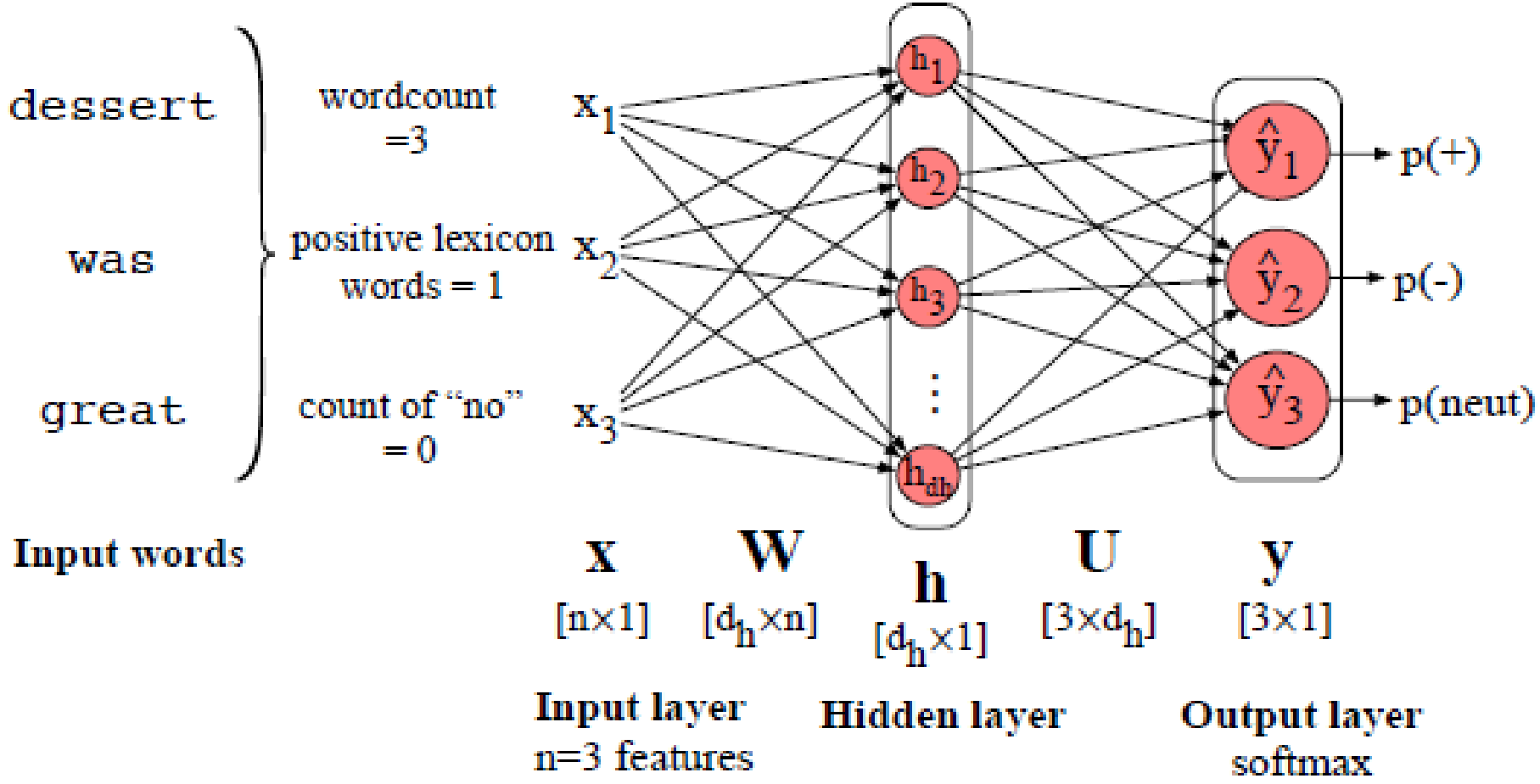
Var	Definition
x_1	count(positive lexicon) $\in$ doc)
x_2	count(negative lexicon) $\in$ doc)
x_3	$\begin{cases} 1 & \text{if } \text{“no”} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$
x_4	count(1st and 2nd pronouns $\in$ doc)
x_5	$\begin{cases} 1 & \text{if } \text{“!”} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$
x_6	log(word count of doc)

Feedforward nets for simple classification



- Just adding a hidden layer to logistic regression
  - allows the network to use non-linear interactions between features
  - could already improve performance (or not).

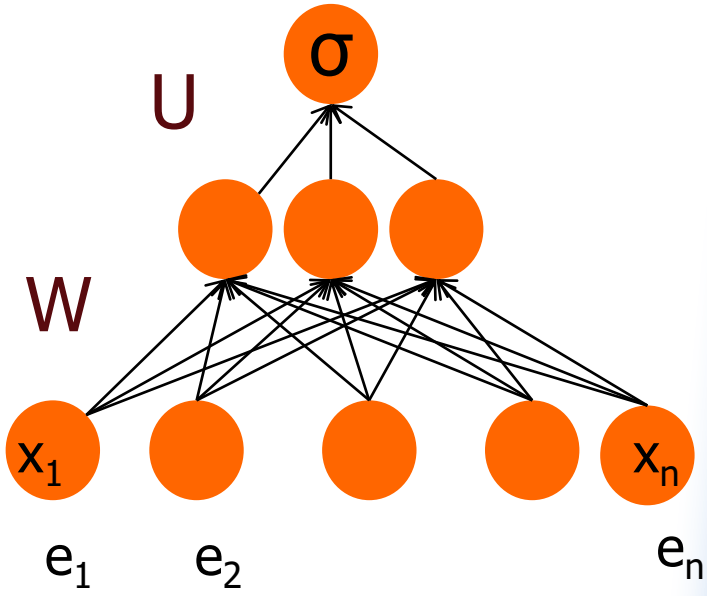
Neural Net Classification with hand-built (Input) Features!



FF net. SA using hand-built features of the input text

Even better: representation learning

- The real power of deep learning comes from the ability to **learn features** from the data
  - Instead of using hand-built human-engineered features as input to the classifier
  - Use learned word representations (like word2vec or GloVe) as embeddings!



Issue: texts come in different sizes

- Some simple solutions (more sophisticated ones later):

1. Make input length = that of the longest review/doc.
   - If shorter then pad with zero embeddings
   - Truncate if you get longer reviews at test time

2. Apply some sort of a **pooling** function to the embeddings of the words:

- E.g. for a text/sentence with n input words/tokens $w_1, \ldots, w_n$, turn the n embeddings $e(w_1), \ldots, e(w_n)$ (each of dimensionality d) into a single d-dimension embedding by:
  - summing the embeddings, or
  - taking their mean: $x_{mean} = \frac{1}{n} \sum_{i=1}^{n} e(w_i)$ or
  - taking their element-wise max.

Neural Net Classification with Embeddings as Input Features!

- Assuming mean pooling of the embeddings and a single example, the equations for the classifier are:

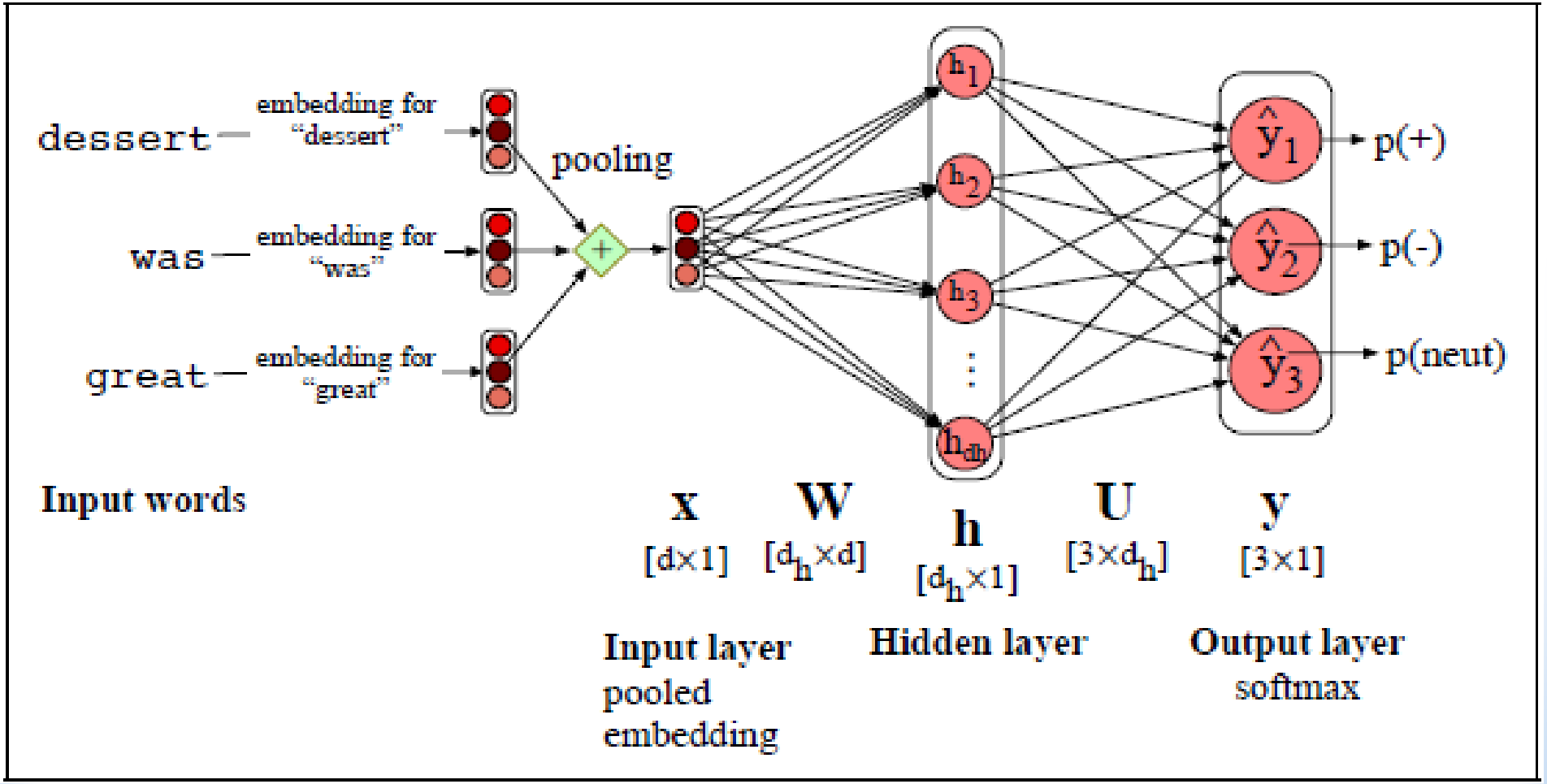
$$x = \text{mean}(e(w_1), \ldots, e(w_n))$$

$$h = \sigma(W.x+b)$$

$$z = U.h$$

$$\hat{y} = \text{softmax}(z)$$

Neural Net Classification with Embeddings as Input Features!



FF net. SA using a pooled embedding of the input words

classifying an entire test set of examples

- Use matrix multiplication to do the computation of an entire test set at once.
  - For each input *x*, pack all the input feature vectors into a single input matrix *X*, with each row *i* a row vector consisting of the pooled embedding for input example *x(i)*.
  - For m examples, *X* is of shape [*m x d*] and W is of shape [d_h x d]; *W* is transposed
    ➔ product will give a matrix [*m x* d_h].
  - Bias vector b of shape [*1x*d_h] is replicated into a matrix of shape [*mx*d_h].
  - Output matrix $\hat{Y}$ is [m x 3] (or more generally [m x d_o], where d_o is the number of output classes),

Output class distribution for a set of examples

$$\mathbf{H} = \sigma(\mathbf{X}\,\mathbf{W}^{\mathbf{T}} + \mathbf{b})$$

$$\mathbf{Z} = \mathbf{U}\,\mathbf{H}$$

$$\hat{\boldsymbol{Y}} = \text{softmax}(\mathbf{Z})$$

- **Pretraining:** The idea of using word2vec or GloVe embeddings as input representation — and more generally the idea of relying on another algorithm to have already learned an embedding representation for our input words.
- Using static embeddings like word2vec or more powerful contextual embeddings (Chap. 11), is one of the central ideas of deep learning.

Feedforward Neural Language Modeling

Neural Language Models

- **Language Modeling**: Calculating the probability of the next word in a sequence given some history.
- We've seen N-gram based LMs
- But neural network LMs far outperform n-gram language models
- State-of-the-art neural LMs are based on more powerful neural network technology like Transformers (Upcoming chapter).
- But **simple feedforward Neural LMs** can do almost as well!

N-gram models vs. neural LMs

- Neural LMs can
  - handle much longer histories,
  - can generalize better over contexts of similar words, and
  - are more accurate at word-prediction.
- But neural net LMs are
  - much more complex,
  - slower and need more energy to train, and
  - less interpretable than n-gram models.
- So for some smaller tasks n-gram LMs are still the right tool.

Simple FF Neural Language Models

- A FF neural LM is a FF net that
  - takes as **input at time t** a representation of some number of previous words (w_{t-1}, w_{t-2}, ...) and
  - **outputs** a probability distribution over possible next words.

Simple FF Neural Language Models

- **Task**: At each timestep *t* predict proba. of next word w_t given prior words $w_{t-1}, w_{t-2}, w_{t-3}, \ldots$
- **Problem**: We are now dealing with sequences of arbitrary length.
- **Solution**: Use sliding windows (of fixed length)

$$P(w_t | w_1^{t-1}) \approx P(w_t | w_{t-N+1}^{t-1})$$

- i.e. the FF neural LM approximates the probability of a word given the entire prior context $P(w_t / w_{1:t-1})$ by approximating based on the $N-1$ previous words.

Simple FF Neural LMs (NLMs)

- Example: we will use a 4-gram case, so we'll show a neural net to estimate the probability $P(w_t = i | w_{t-3}, w_{t-2}, w_{t-1})$.
- NLMs represent words in this prior context by their embeddings, rather than just by their word identity as used in n-gram LMs.
- This allows them to generalize better to unseen data.

Why Neural LMs work better than N-gram LMs

- Suppose:
- **Training data:**
  - We've seen:

    I have to make sure that the cat gets fed.
  - Never seen: *dog gets fed*
- **Test data:**
  - *I forgot to make sure that the dog gets ___*
  - N-gram LMs can't predict "*fed*"!
  - Neural LMs can use similarity of "*cat*" and "*dog*" embeddings to generalize and predict "*fed*" after *dog*

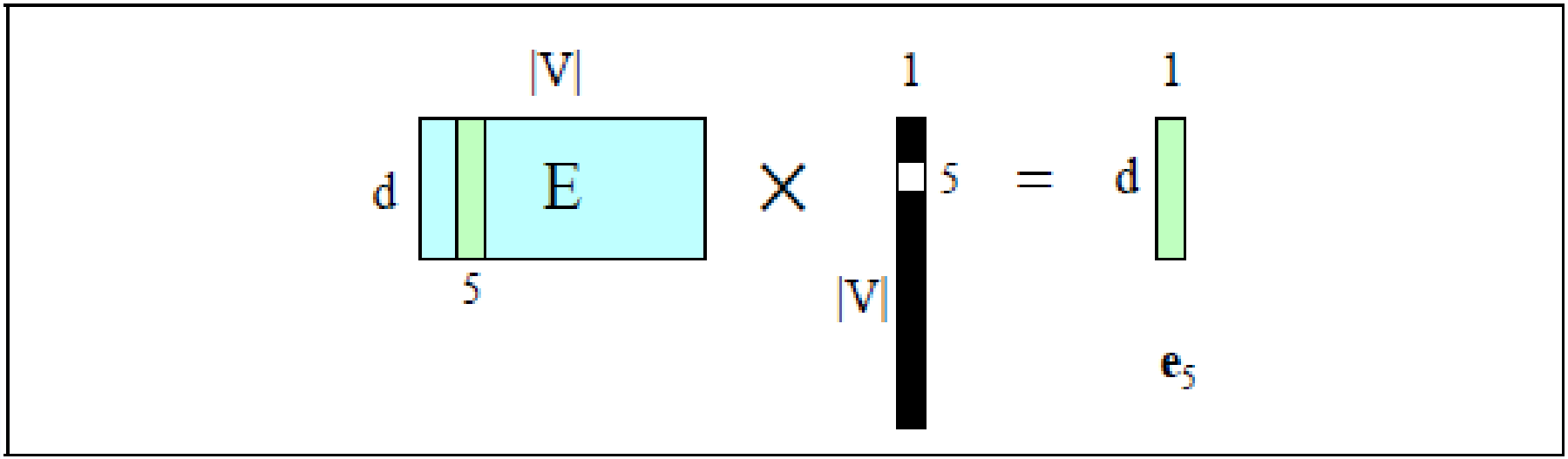
Forward inference in a FF NLM

- **Forward inference**: given an input, run a forward pass on the network to produce a probability distribution over possible outputs, in this case next words.

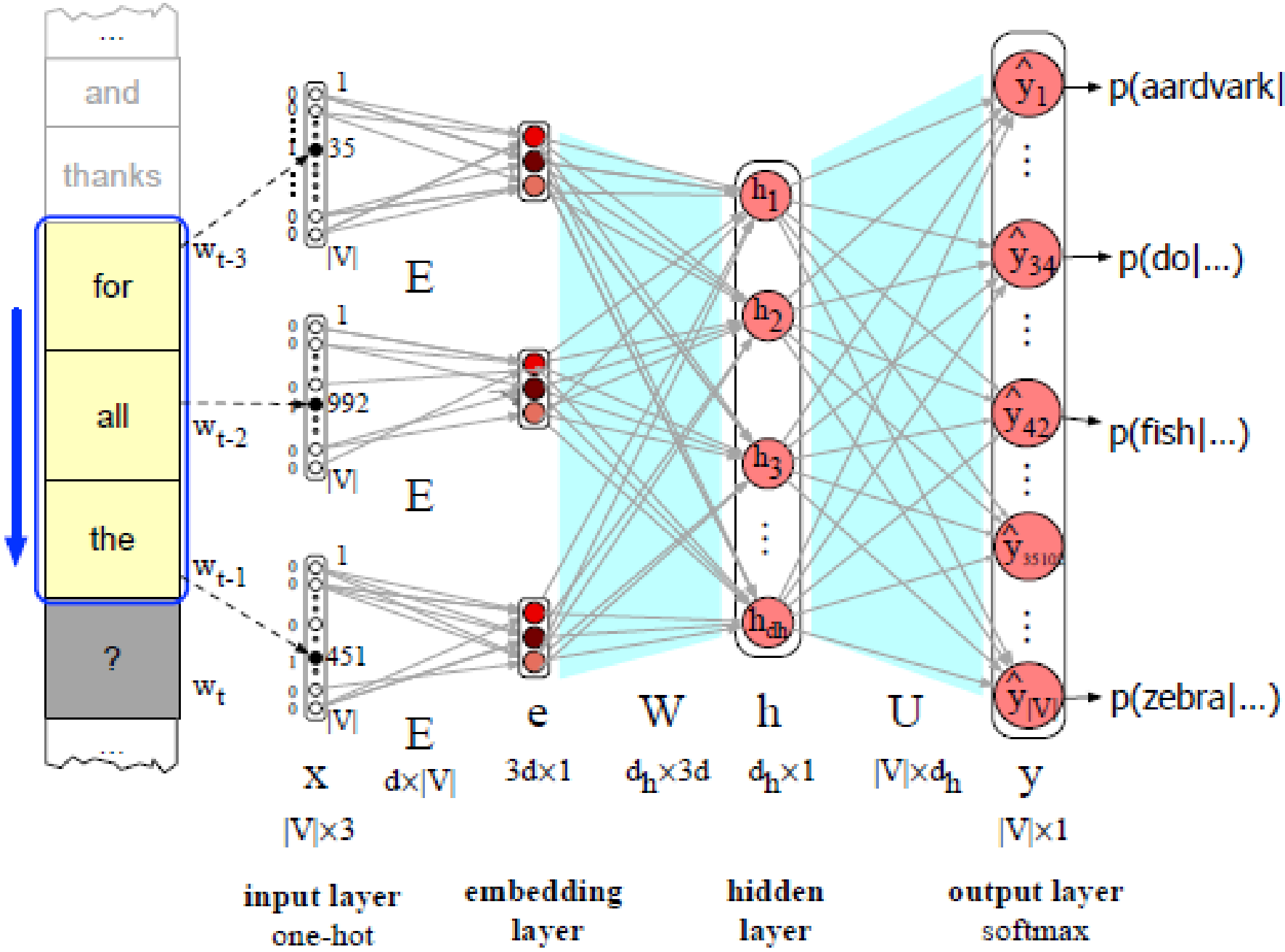
1. Represent each of the *N* previous words as a **one-hot vector** of length *|V|*.
2. Multiply these one-hot vectors by the embedding matrix E.
   - ***E*** has a column for each word, each being a column vector of **d** dimensions
     ➔ ***E*** has dimensionality *d x |V|*

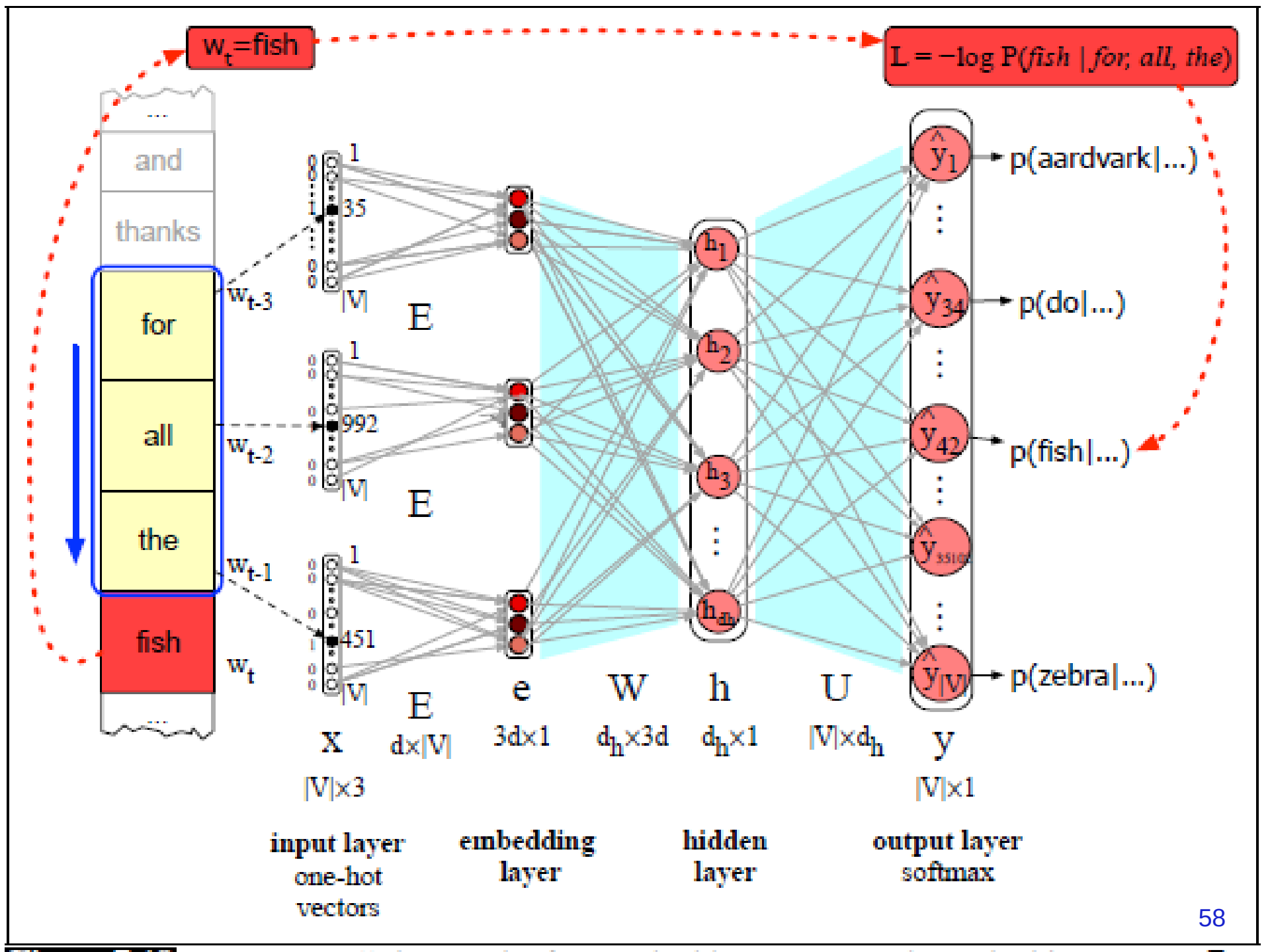
Forward inference in a FF NLM

Multiplying by a one-hot vector that has only one non-zero element $x_i = 1$ simply selects the relevant column vector for word *i*, resulting in the embedding for word i.



The 3 resulting embedding vectors are concatenated to produce *e*, the embedding layer. Size *3d x 1*


...
and
thanks
for
all
the
?
...
w_{t-3}
w_{t-2}
w_{t-1}
w_t
1
35
|V|
992
451
E
E
E
h_1
h_2
h_3
h_{dh}
$\hat{y}_1$
$\hat{y}_{34}$
$\hat{y}_{42}$
$\hat{y}_{|V|}$
p(aardvark|...)
p(do|...)
p(fish|...)
p(zebra|...)
x
|V|×3
d×|V|
e
3d×1
W
d_h×3d
h
d_h×1
U
|V|×d_h
y
|V|×1
input layer
one-hot
vectors
embedding
layer
hidden
layer
output layer
softmax


w_t=fish
L = −log P(*fish* | *for, all, the*)
...
and
thanks
for
all
the
fish
...
w_{t-3}
w_{t-2}
w_{t-1}
w_t
1
35
|V|
1
992
|V|
1
451
|V|
E
E
E
$\hat{y}_1$
$\hat{y}_{34}$
$\hat{y}_{42}$
$\hat{y}_{|V|}$
p(aardvark|...)
p(do|...)
p(fish|...)
p(zebra|...)
h_1
h_2
h_3
h_{d_h}
x
d×|V|
e
3d×1
W
d_h×3d
h
d_h×1
U
|V|×d_h
y
|V|×1
|V|×3
input layer
one-hot
vectors
embedding
layer
hidden
layer
output layer
softmax

Example

- Y_{42} (value of output node 42): probability of the next word w_t being V_{42}, the vocabulary word with index 42 (the word 'fish' in this case)

1. Select three embeddings from *E* :
   - Given the three previous words, look up their indices, create 3 one-hot vectors, and then multiply each by the embedding matrix *E*.
   - Consider $w_{t-3,}$ one-hot vector for '*for*' (index 35) is multiplied by the embedding matrix *E*, to give the first part of the first hidden layer, the **embedding layer**

- Recall: - each column of the input matrix *E* is an embedding for a word, &
  - the input is a one-hot column vector x_i for word V_i,

  ➔ embedding layer for input w will be $Ex_i = e_i$, the embedding for word *i*.
   - Concatenating the 3 embeddings for the 3 context words we get the embedding layer *e*.

Example (cont.)

2. Multiply by W (and add b) and pass through the ReLU (or other) activation function to get the hidden layer h.
3. Multiply h by U
4. Apply softmax: After the softmax, each node i in the output layer estimates the probability $P(w_t = i | w_{t-3}, w_{t-2}, w_{t-1})$.

Summary Equations

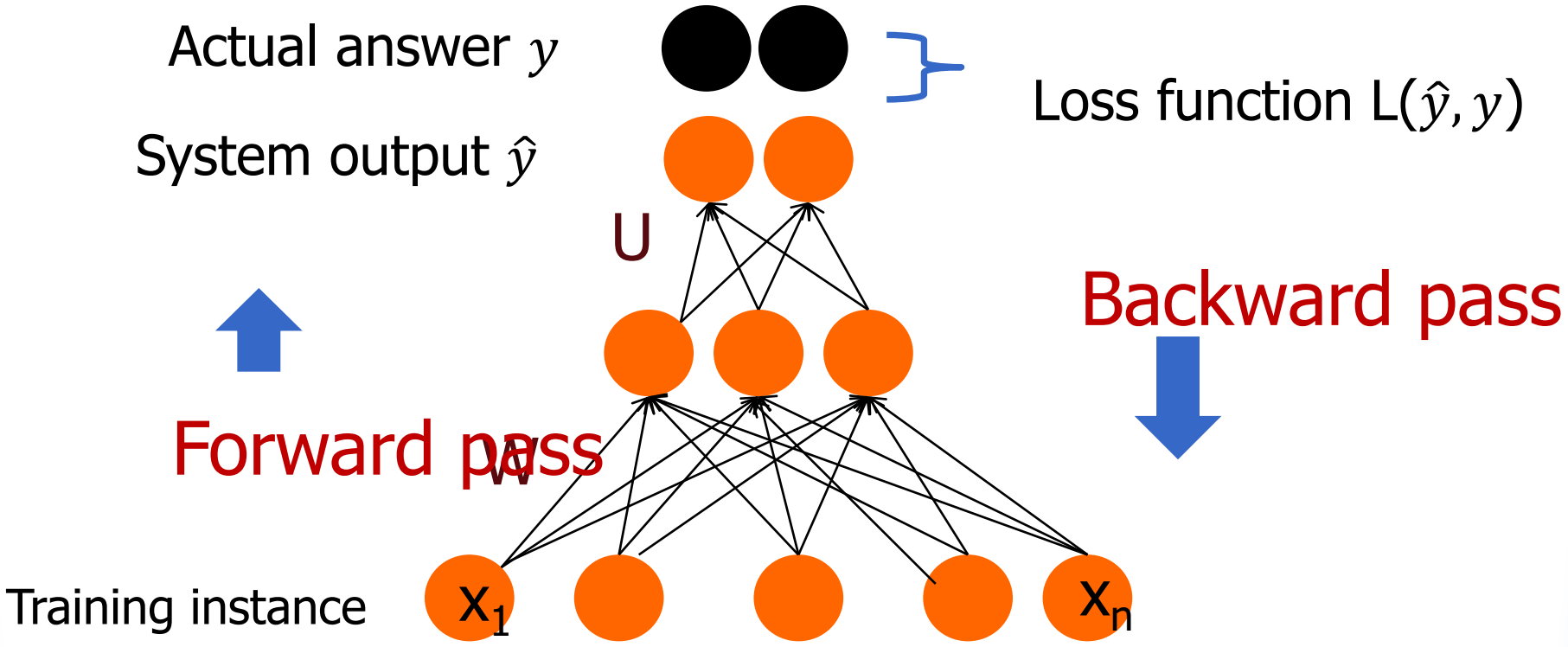
- The equations for a NLM with a window size of 3, given one-hot input vectors for each input context word, are:
- $e = [E\, x_{t-3}; E\, x_{t-2}; E\, x_{t-1}]$

  # concatenation of 3 embeddings
- $h = \sigma(We+b)$
- $z = Uh$
- $\hat{y} = \text{softmax}(z)$

Training Neural Nets: Overview

(Material from ML: Individual Reading)

Intuition: training a 2-layer Network



Intuition: Training a 2-layer network

- For every training tuple (x, y)
  - Run *forward* computation to find our estimate $\hat{y}$
  - Run *backward* computation to update weights:
    - For every output node
      - Compute loss L between true y and the estimated $\hat{y}$
      - For every weight w from hidden layer to the output layer
    - Update the weight
    - For every hidden node
      - Assess how much blame it deserves for the current answer
      - For every weight w from input layer to the hidden layer
    - Update the weight

Reminder: Loss Function for binary logistic regression

- A measure for how far off the current answer is to the right answer
- Cross entropy loss for logistic regression:

$$L_{\text{CE}}(\hat{y}, y) = -\log p(y|x) = -[y \log \hat{y} + (1-y)\log(1-\hat{y})]$$

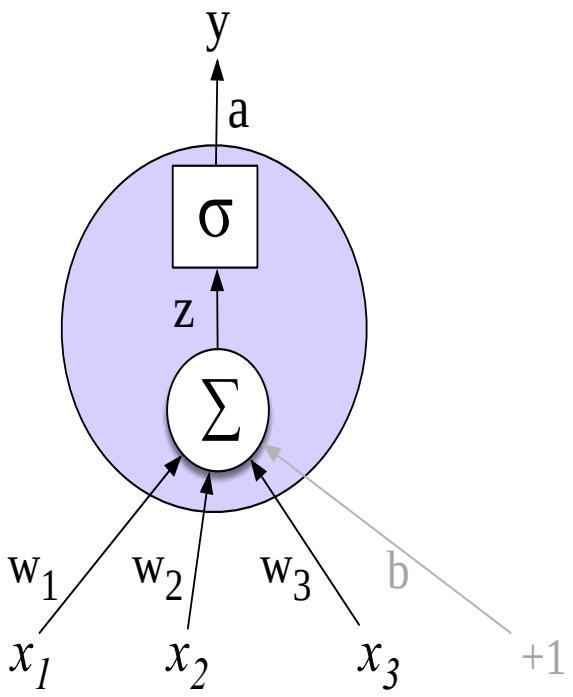
$$= -[y \log \sigma(w \cdot x + b) + (1-y)\log(1-\sigma(w \cdot x + b))]$$

Reminder: gradient descent for weight updates

- Use the derivative of the loss function with respect to weights $\frac{d}{dw} L(f(x;w),y)$
- To tell us how to adjust weights for each training item
  - Move them in the opposite direction of the gradient $$w^{t+1} = w^t - h \frac{d}{dw} L(f(x;w),y)$$
  - For logistic regression $$\frac{\partial L_{\text{CE}}(\hat{y},y)}{\partial w_j} = [\sigma(w \cdot x + b) - y] x_j$$

Derivative of a composition of functions

- Using the chain rule! $f(x) = u(v(x))$
- Intuition (see the text for details) $\frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dx}$



Derivative of the Activation

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial y}\frac{\partial y}{\partial z}\frac{\partial z}{\partial w_i}$$

Derivative of the Loss

Derivative of the weighted sum

Finding the gradient for every weight in the network

- These derivatives on the prior slide only give the updates for one weight layer: the last one!
- What about deeper networks?
- Lots of layers, different activation functions?
- Solution in the next lecture:
  - Even more use of the chain rule!!
  - Computation graphs and backward differentiation!

Computation Graphs and Backward Differentiation/ Backpropagation

(Basic ML material: individual reading)

Why Computation Graphs

- For training, we need the derivative of the loss with respect to each weight in every layer of the network
- But the loss is computed only at the very end of the network!
- Solution: **error backpropagation** (Rumelhart, Hinton, Williams, 1986)
- **Backprop** is a special case of **backward differentiation**
- Which relies on **computation graphs**.

Computation Graphs

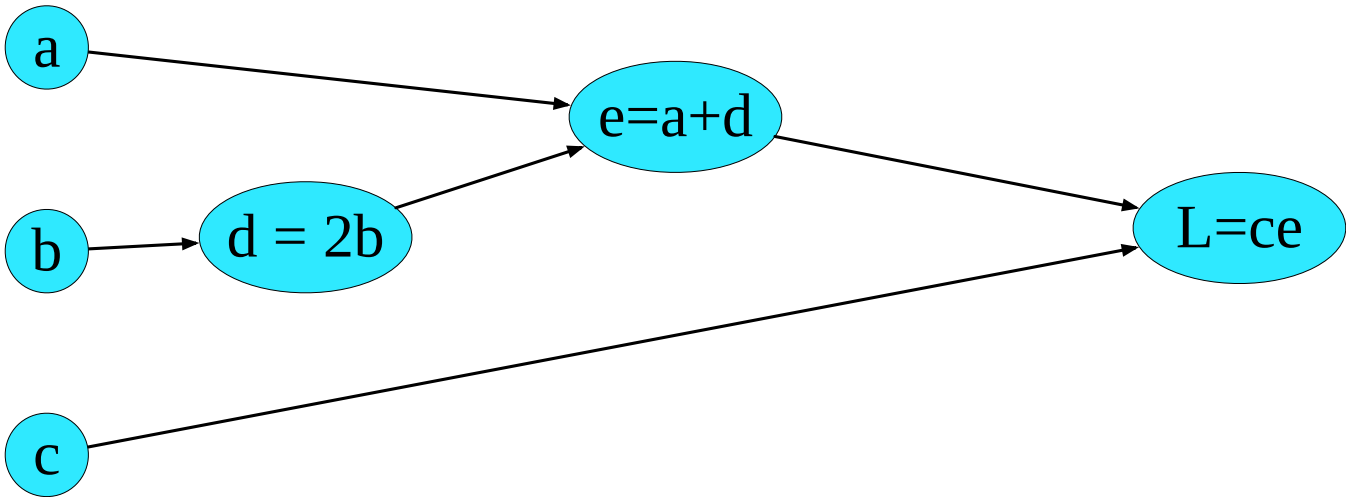
- A computation graph represents the process of computing a mathematical expression

Example:

$$L(a,b,c) = c(a+2b)$$

Computations:

$$d = 2*b$$
$$e = a+d$$
$$L = c*e$$

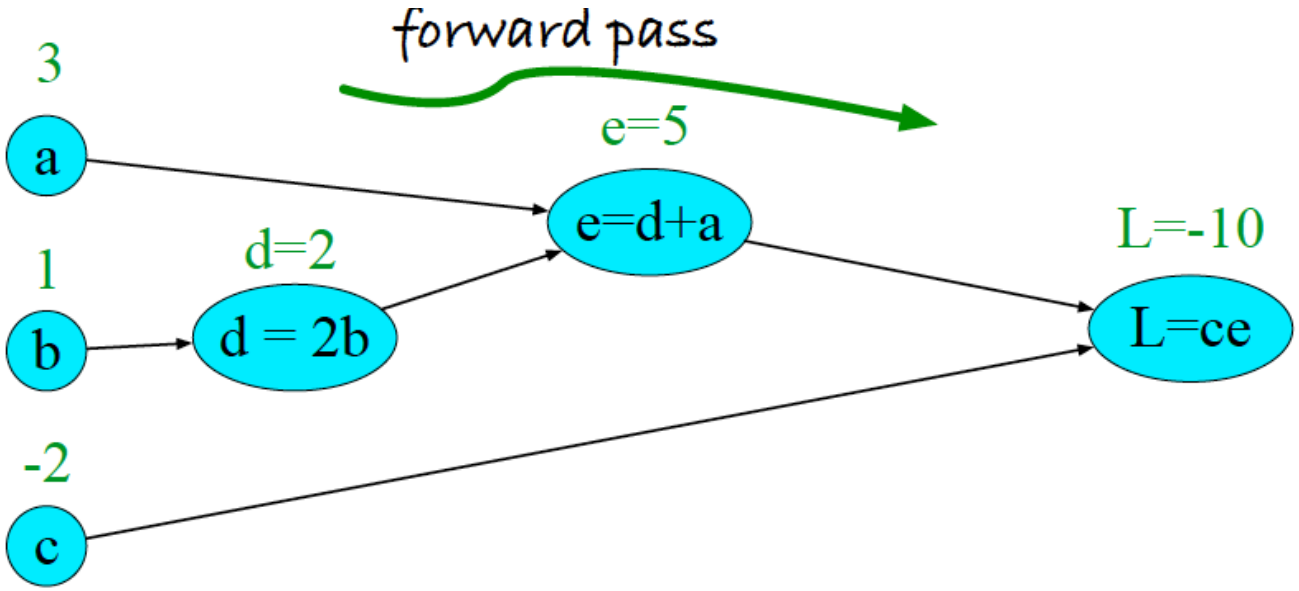


Example:

$$L(a,b,c) = c(a+2b)$$

Computations:

$$d = 2*b$$
$$e = a+d$$
$$L = c*e$$



Backwards differentiation in computation graphs

- The importance of the computation graph comes from the backward pass
- This is used to compute the derivatives that we'll need for the weight update.

Example

$$L(a,b,c) = c(a+2b)$$

$$\begin{aligned} d &= 2*b \\ e &= a+d \\ L &= c*e \end{aligned}$$

We want: $\frac{\partial L}{\partial a}$, $\frac{\partial L}{\partial b}$, and $\frac{\partial L}{\partial c}$

The derivative $\frac{\partial L}{\partial a}$, tells us how much a small change in *a* affects *L*.

The chain rule

- Computing the derivative of a composite function:

- $f(x) = u(v(x))$ $\qquad \frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dx}$

- $f(x) = u(v(w(x)))$ $\qquad \frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dw} \cdot \frac{dw}{dx}$

Example

$$L(a,b,c) = c(a+2b)$$

$$\begin{aligned} d &= 2*b \\ e &= a+d \\ L &= c*e \end{aligned}$$

$$\begin{aligned} \frac{\partial L}{\partial c} &= e \\ \frac{\partial L}{\partial a} &= \frac{\partial L}{\partial e}\frac{\partial e}{\partial a} \\ \frac{\partial L}{\partial b} &= \frac{\partial L}{\partial e}\frac{\partial e}{\partial d}\frac{\partial d}{\partial b} \end{aligned}$$

Example

$$L(a,b,c) = c(a+2b)$$

$$\begin{aligned} d &= 2*b \\ e &= a+d \\ L &= c*e \end{aligned}$$

$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial e}\frac{\partial e}{\partial a}$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial e}\frac{\partial e}{\partial d}\frac{\partial d}{\partial b}$$

$$L = ce : \quad \frac{\partial L}{\partial e} = c, \frac{\partial L}{\partial c} = e$$

$$e = a+d : \quad \frac{\partial e}{\partial a} = 1, \frac{\partial e}{\partial d} = 1$$

$$d = 2b : \quad \frac{\partial d}{\partial b} = 2$$

Example

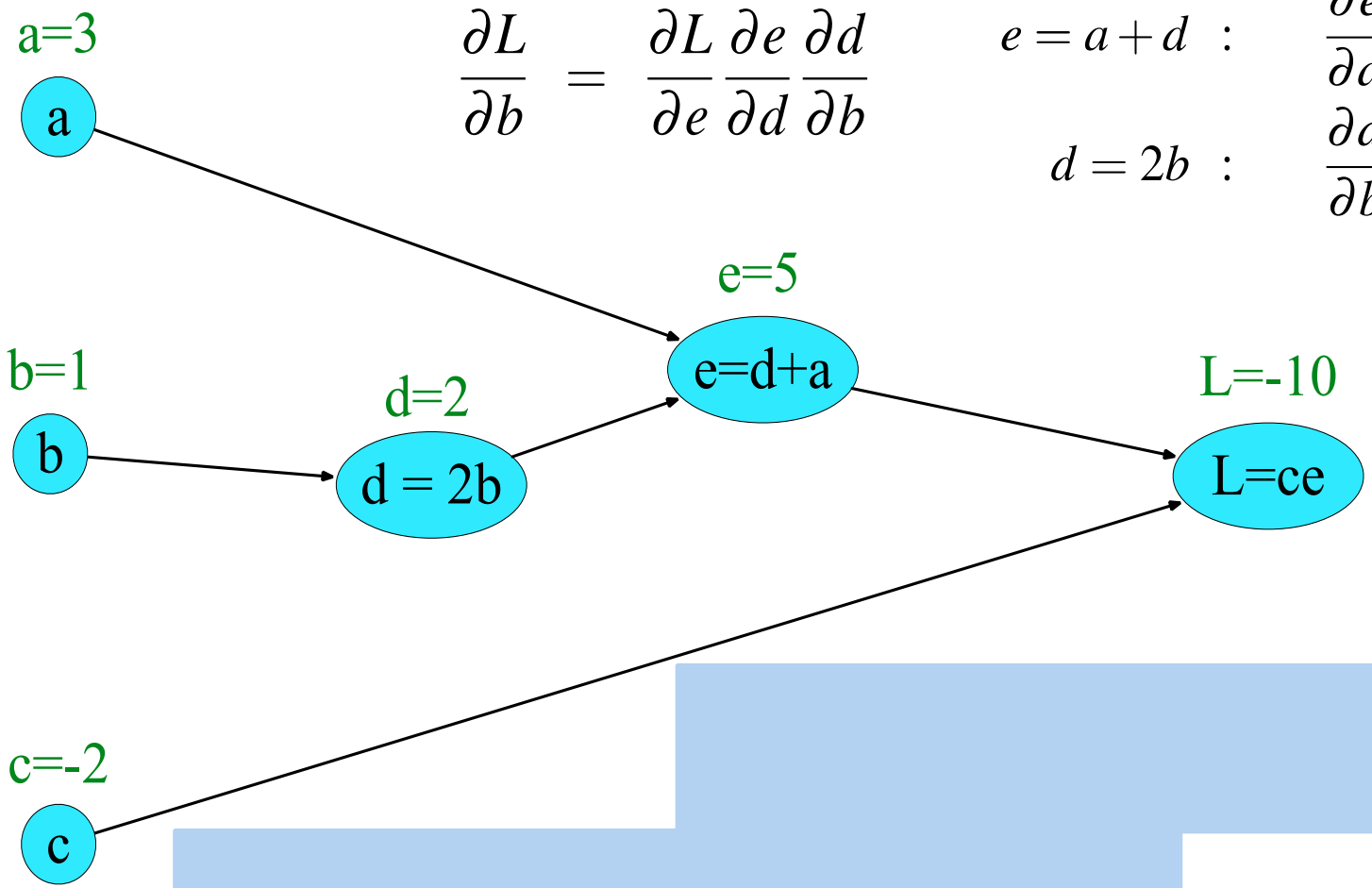
$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial e}\frac{\partial e}{\partial a}$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial e}\frac{\partial e}{\partial d}\frac{\partial d}{\partial b}$$

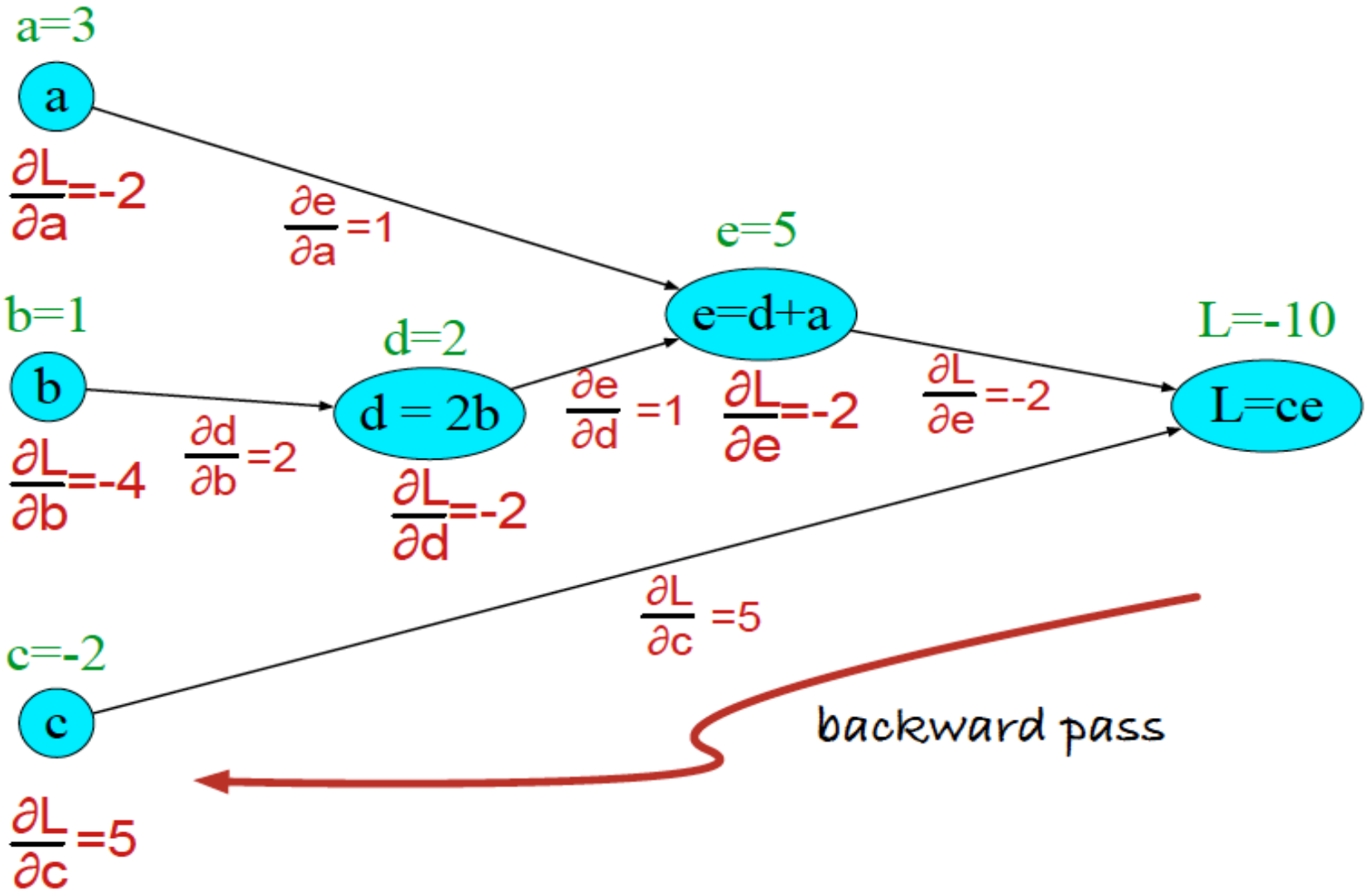
$$L = ce : \quad \frac{\partial L}{\partial e} = c, \frac{\partial L}{\partial c} = e$$

$$e = a + d : \quad \frac{\partial e}{\partial a} = 1, \frac{\partial e}{\partial d} = 1$$

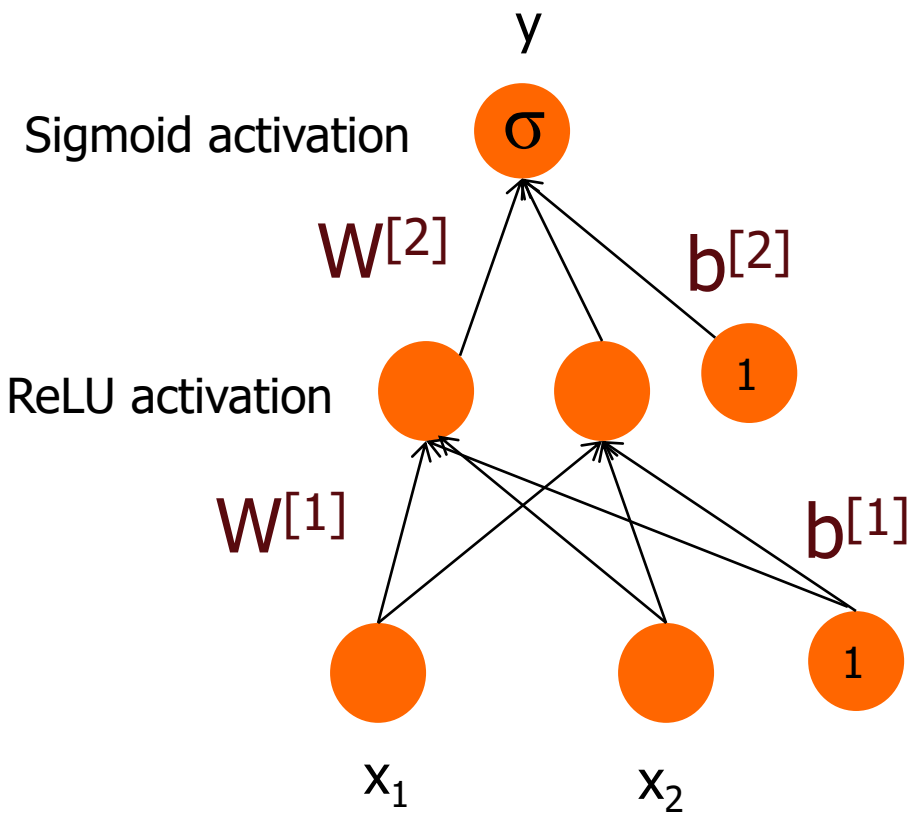
$$d = 2b : \quad \frac{\partial d}{\partial b} = 2$$



Example


a=3
a
∂L/∂a=-2
∂e/∂a=1
e=5
e=d+a
b=1
b
∂L/∂b=-4
∂d/∂b=2
d=2
d = 2b
∂e/∂d=1
∂L/∂d=-2
∂L/∂e=-2
L=-10
L=ce
∂L/∂c=5
c=-2
c
backward pass
∂L/∂c=5

Backward differentiation on a 2-layer network



$$z^{[1]} = W^{[1]}\mathbf{x} + b^{[1]}$$
$$a^{[1]} = \text{ReLU}(z^{[1]})$$
$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$
$$a^{[2]} = \sigma(z^{[2]})$$
$$\hat{y} = a^{[2]}$$

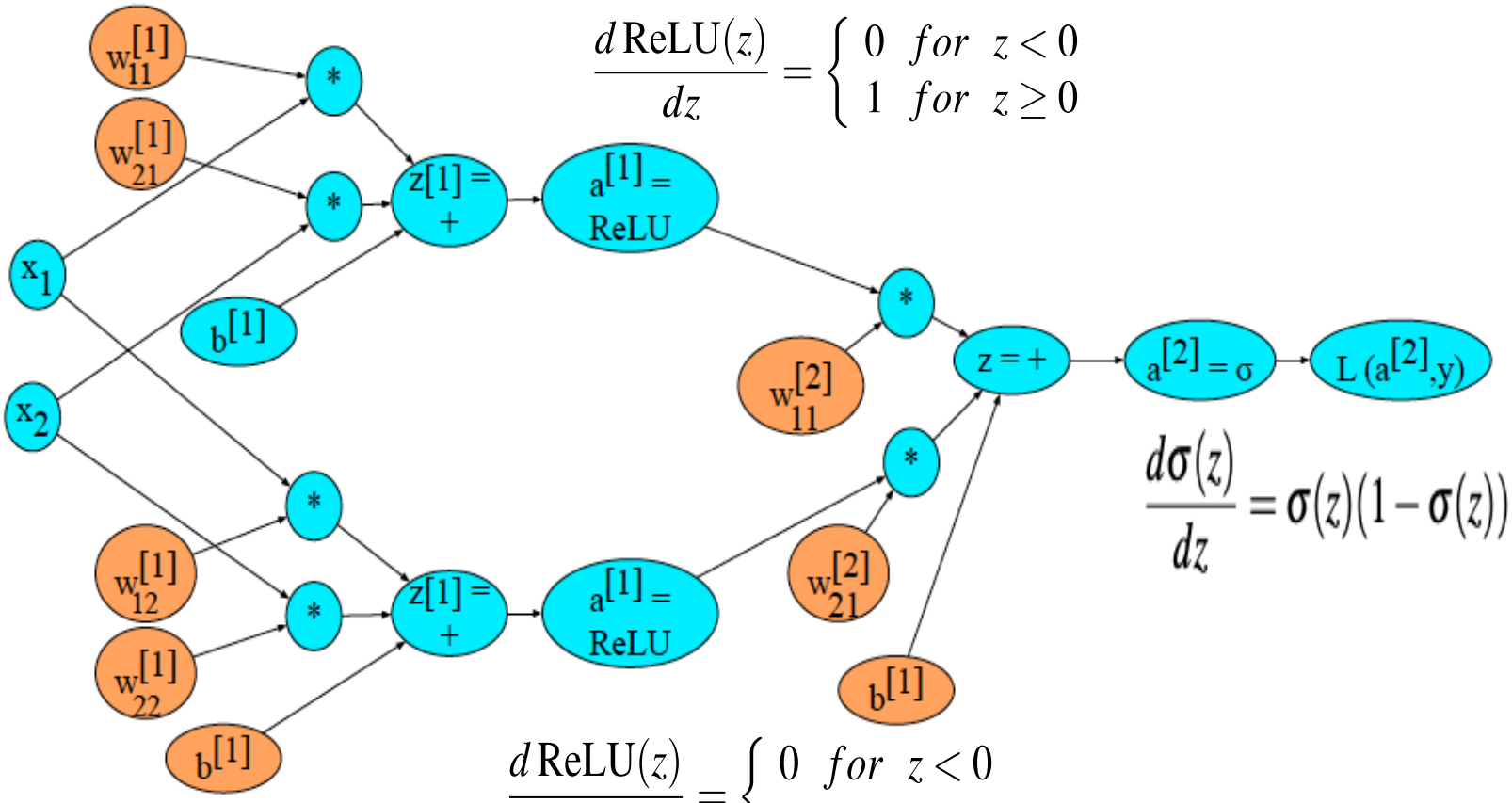
Backward differentiation on a 2-layer network

$$
\begin{aligned}
z^{[1]} &= W^{[1]}\mathbf{x} + b^{[1]} \\
a^{[1]} &= \text{ReLU}(z^{[1]}) \\
z^{[2]} &= W^{[2]}a^{[1]} + b^{[2]} \\
a^{[2]} &= \sigma(z^{[2]}) \\
\hat{y} &= a^{[2]}
\end{aligned}
$$

$$
\frac{d\,\text{ReLU}(z)}{dz} = \begin{cases} 0 & for \;\; z < 0 \\ 1 & for \;\; z \geq 0 \end{cases}
$$

$$
\frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z))
$$

Backward differentiation on a 2-layer network



Starting off the backward pass: $\frac{\partial L}{\partial z}$

(We'll write a for $a^{[2]}$ and z for $z^{[2]}$)

$$L(\hat{y}, y) = -(\mathrm{y}\log(\hat{y}) + (1-y)\log(1-\hat{y}))$$

$$L(a, y) = -(\mathrm{y}\log a + (1-y)\log(1-a))$$

$$\begin{aligned} z^{[1]} &= W^{[1]}\mathbf{x} + b^{[1]} \\ a^{[1]} &= \mathrm{ReLU}(z^{[1]}) \\ z^{[2]} &= W^{[2]}a^{[1]} + b^{[2]} \\ a^{[2]} &= \sigma(z^{[2]}) \\ \hat{y} &= a^{[2]} \end{aligned}$$

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial a}\frac{\partial a}{\partial z}$$

$$\frac{\partial L}{\partial a} = -\left(\left(y\frac{\partial \log(a)}{\partial a}\right) + (1-\mathrm{y})\ \frac{\partial \log(1-a)}{\partial a}\right)$$

$$= -\left(\left(y\frac{1}{a}\right) + (1-\mathrm{y})\ \frac{1}{1-a}(-1)\right) = -\left(\frac{y}{a} + \frac{y-1}{1-a}\right)$$

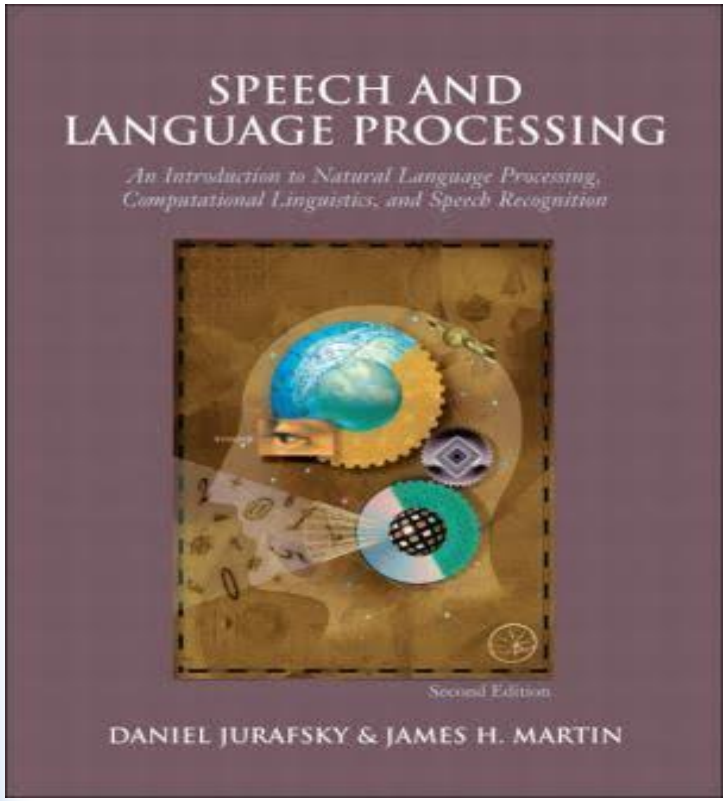
$$\frac{\partial a}{\partial z} = a(1-a) \qquad \frac{\partial L}{\partial z} = -\left(\frac{y}{a} + \frac{y-1}{1-a}\right)a(1-a) = \mathrm{a} - \mathrm{y}$$

Summary

- For training, we need the derivative of the loss with respect to weights in early layers of the network
- But loss is computed only at the very end of the network!
- Solution: **backward differentiation (Backpropagation)**
- Given a computation graph and the derivatives of all the functions in, it we can automatically compute the derivative of the loss with respect to these early weights.

Textbook

Jurafsky, D. and Martin, J.H. (2024) *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models* (3rd ed. draft), Prentice Hall.
https://web.stanford.edu/~jurafsky/slp3/ed3bookaug20_2024.pdf



The draft version of the 3rd edition will be used; it has been updated to include more recent topics such as Vector Semantics and Embeddings, ANNs and Deep Learning for NLP, Transformers, etc.

This course **slides** are largely based on this textbook and its authors' slides with light customisations whenever seen fit.